

Multidimensional Indexing

Prof. Paolo Ciaccia

DISI – Università di Bologna

Milano, December 1, 2023



What is Multidimensional Indexing?

Data
Queries



Space vs Data partitioning

Grid File
R-tree



Metric indexing

Principles
M-tree

Outline

Multidimensional data

- Everything started in the 70's
 - Cartography & Geographic Information Systems (GIS)
 - Very Large Scale Integration (VLSI) & Computer-Aided Design (CAD)
- New research wave in the 90's
 - Multimedia databases (images, videos, time series, ...)
 - Data mining/analytics
 - DBMS's with spatial and Multi-Media (MM) "extenders"

- Also, a case for classical (relational) DBS
 - Multiple (numerical) attributes
- ## Query by Image and Video Content: The QBIC System

Myron Flickner, Harpreet Sawhney, Wayne Niblack, Jonathan Ashley, Qian Huang, Byron Dom, Monika Gorkani, Jim Hafner, Denis Lee, Dragutin Petkovic, David Steele, and Peter Yanker

IBM Almaden Research Center

Examples

- Geographic/Spatial Information Systems

- Points: places, cities, ...
- Objects with spatial extension: regions, streets, rivers, ...

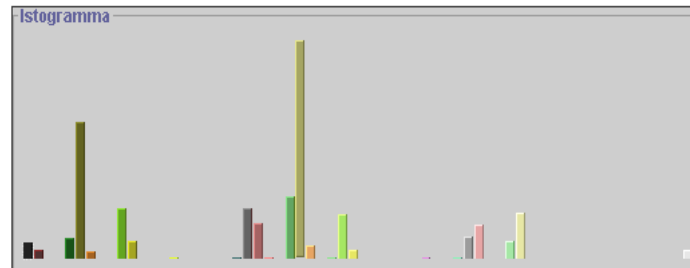


- Multimedia Databases

- Content of MM objects represented by numerical characteristics (*features*)
- Similarity of content = similarity of features



$d = 64$



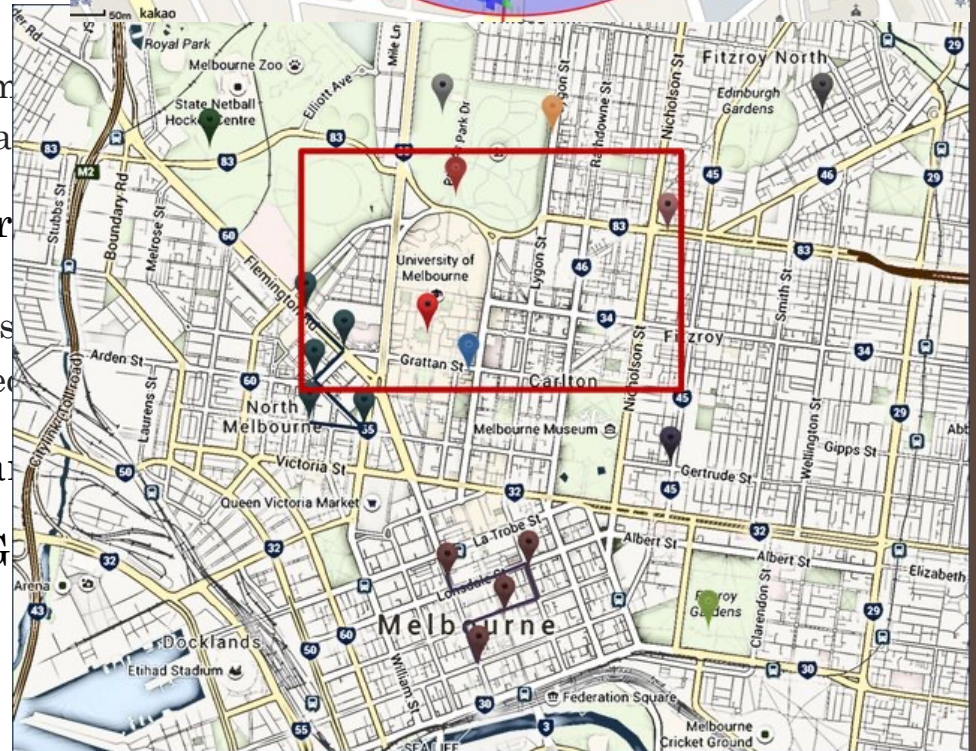
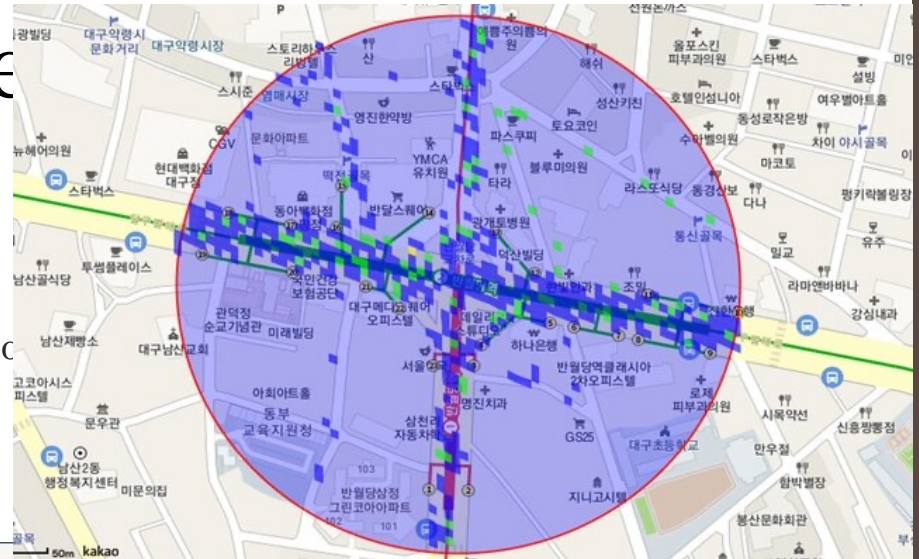
- Relational Databases

- Best combination of multiple attributes
 - According to a user/system criterion
 - E.g., Points + Rebounds

Name	Points	Rebounds
Shaquille O'Neal	1669	760
Tracy McGrady	2003	484
Kobe Bryant	1819	392
Yao Ming	1465	669
Dwyane Wade	1854	397
Steve Nash	1165	249
...	...	...

Basic Query Type

- Point/lookup query
 - What is here?
 - Is this a duplicate object (image, video)
- Window query (= rectangular q.)
 - $\text{Price} \in [10k, 25k]$ & $\text{Km} \in [0, 40k]$
- Range query
 - All pubs in a radius of 1km from m
 - All gas stations at most 10km away
- **k-nearest neighbors (k-NN)** query
 - The 5 closest restaurants
 - The 10 most similar images to this
 - The 20 used cars best fitting a spec
- Both range and k-NN queries can be
- Plus (many) others peculiar to G
 - Streets crossing a railroad
 - Buildings close to a river



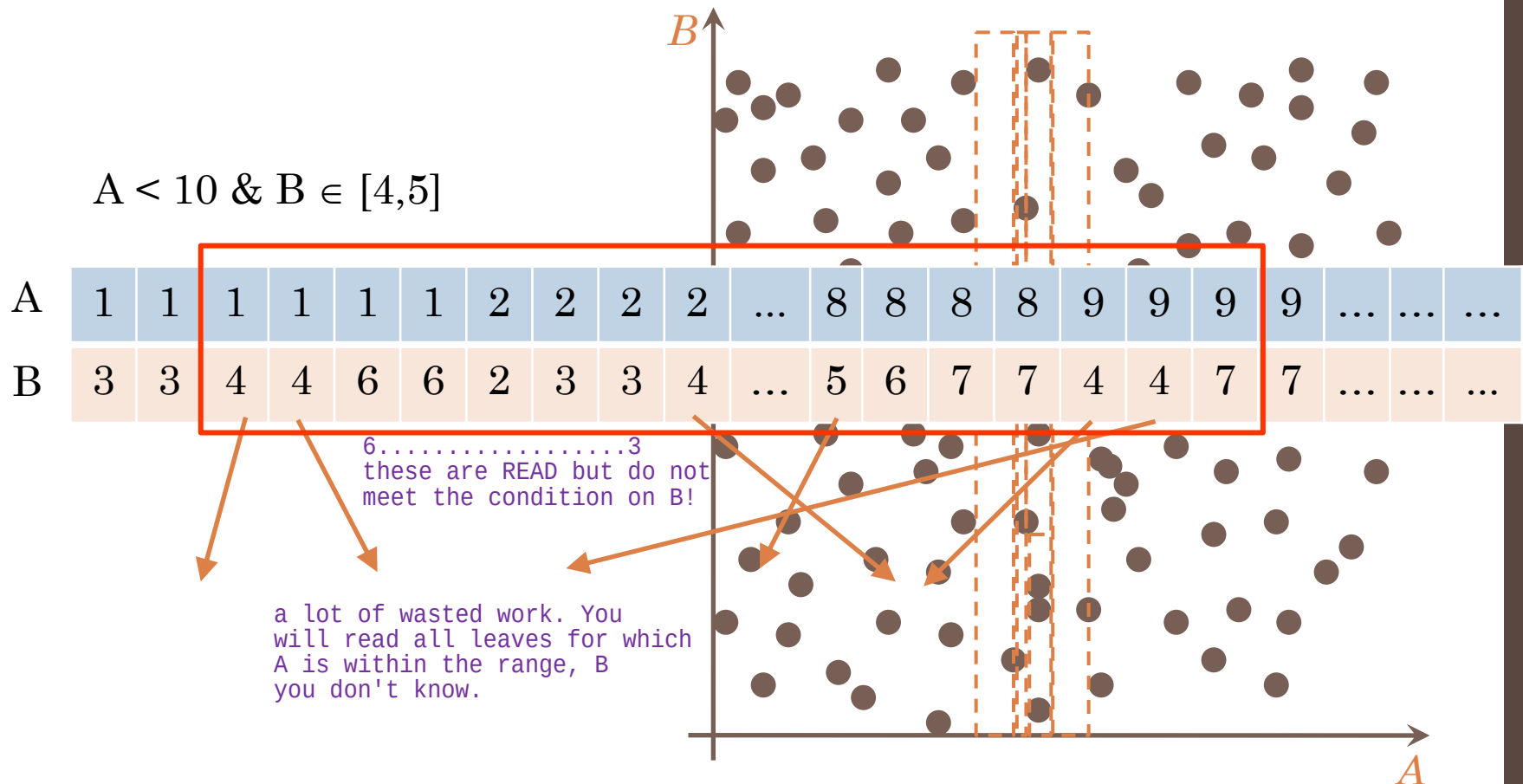
Using B+-trees

- Consider a window query on 2 attributes A and B
 - Assume that every query interval selects 10% of the total n. of points
 - With independent attributes we expect to retrieve 1% of data
- Possible solutions:
 - 1 B+-tree on (A,B) → first sort on A, then on B
 - 2 B+-trees on A and on B

both solutions do not work very well

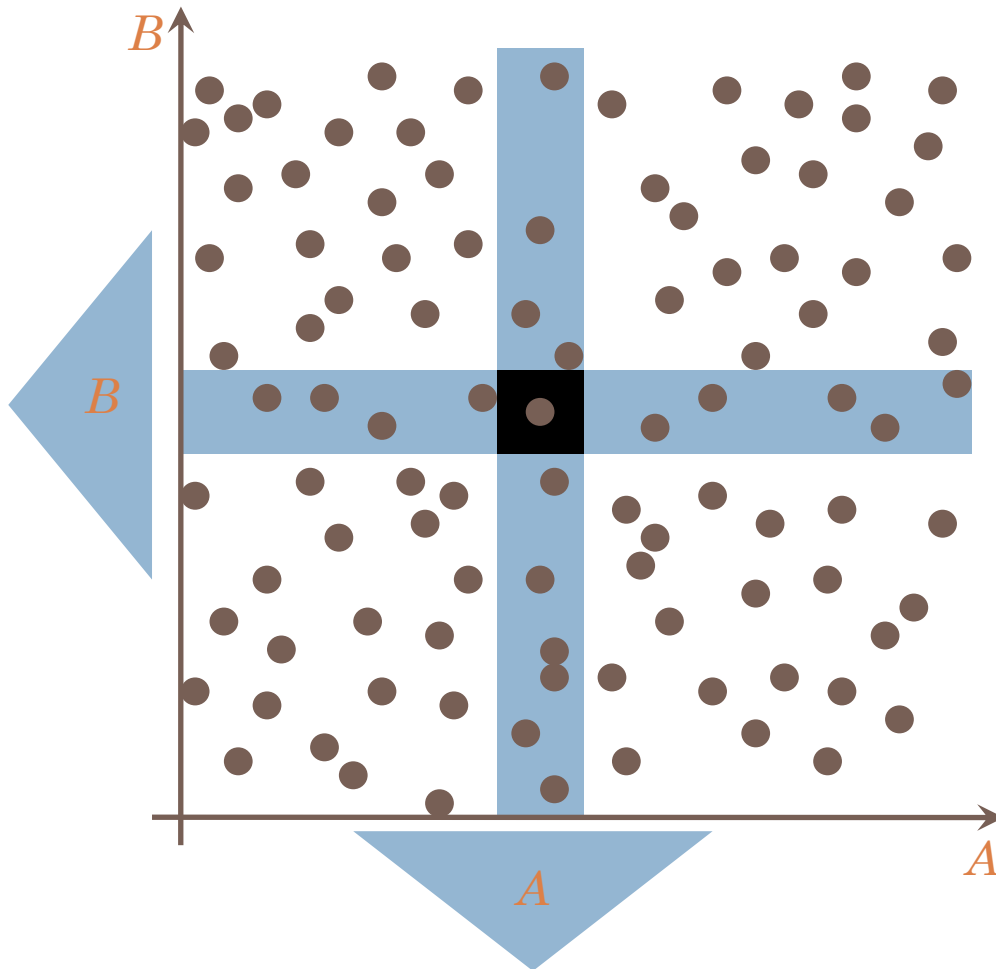
2-dimensional B+-tree (A,B)

- Reads many leaves that do not contain any point in the result
 - Index on (Price,Km), q: Price $\in [10k,30k]$ & Km $\in [30k,40k]$
 - Also reads leaves with Km $\notin [30k, 40k]$



Two 1-dimensional B+-trees

- Access 10% of leaves on both indices
 - RID intersection needed to avoid reading 20% of data



B+-tree efficiency

- In both cases, too much wasted work
- The reason is that points close in space are stored in distant leaves
 - In the first case, due to the “linearization” of attributes
 - In the other case, because we ignore the other attribute(s)
- With more dimensions performance is even worse!
- Multidimensional indices try to preserve the spatial proximity of records (**Local Order Preservation**)

Group objects (points) in pages, guaranteeing that each page contains objects which are “close” in the d -dim space

- The problem is not trivial, since in d dimensions a global order is not defined

Aside: space filling curves

Hilbert's space filling curve

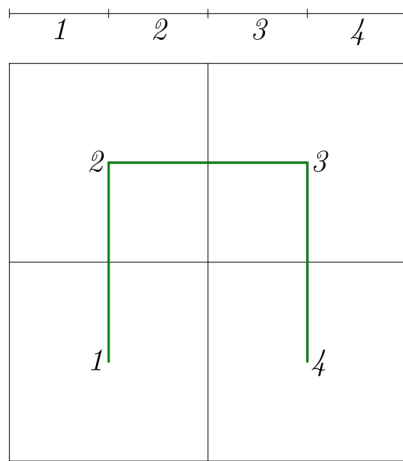


Fig. 1.

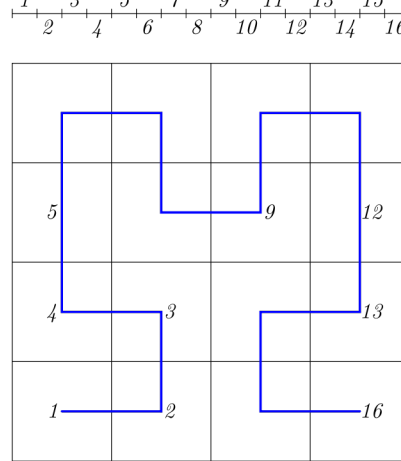


Fig. 2.

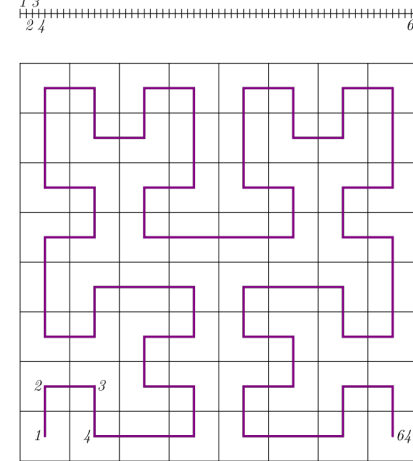
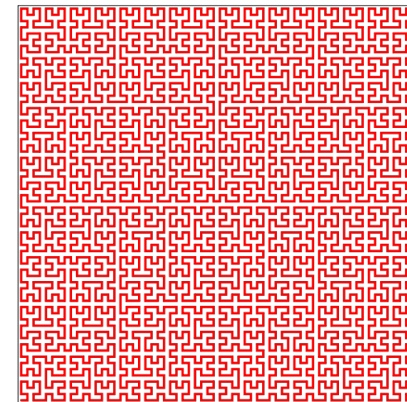
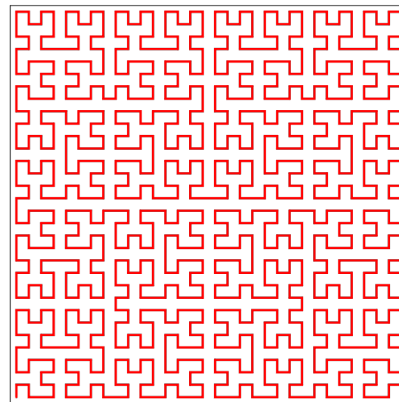
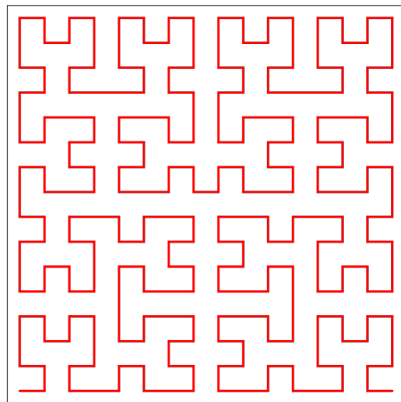


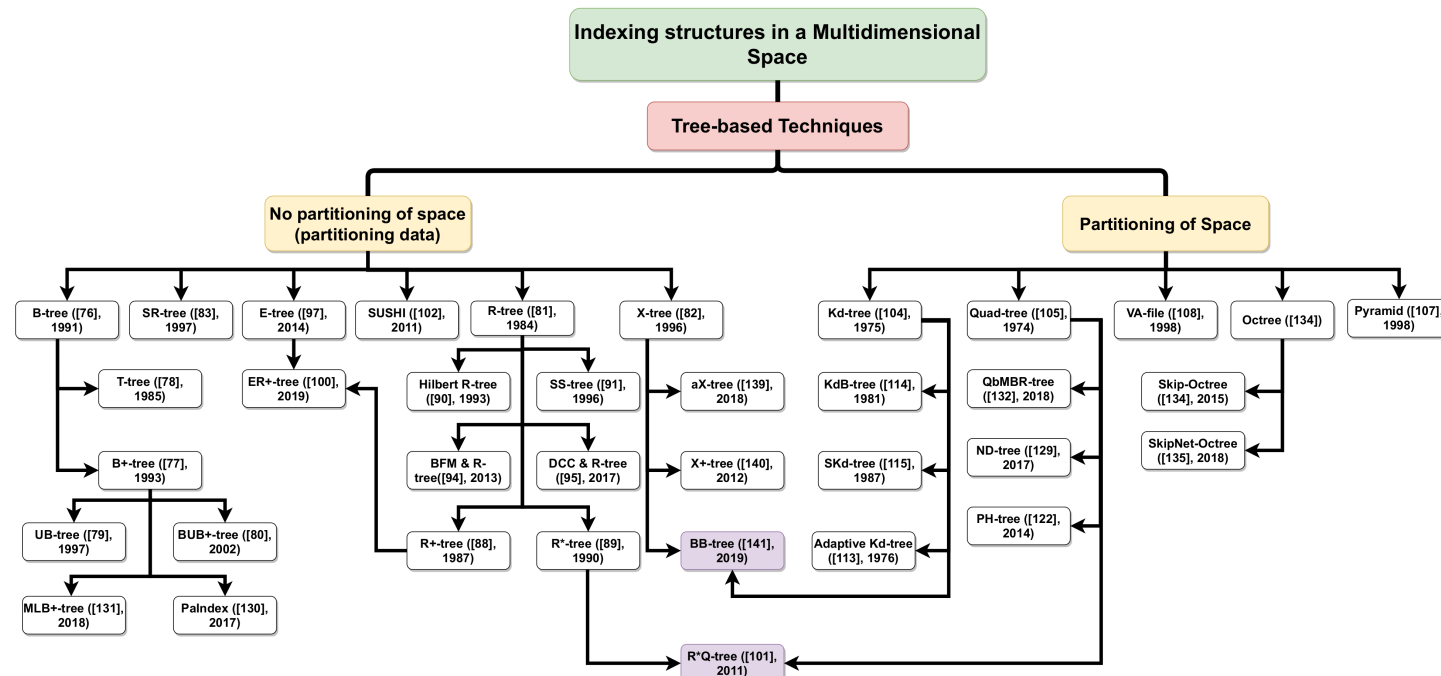
Fig. 3.



They provide a global order but fail to preserve distances

Spatial indices: different approaches

- Derived by 1-dimensional structures: Grid file (and many others)
- rectangle-tree, inspired by B+ trees
- Ad-hoc structures: **R-tree** (its variants R*-tree, R+-tree, and many others)
- In total: **hundreds** of data structures



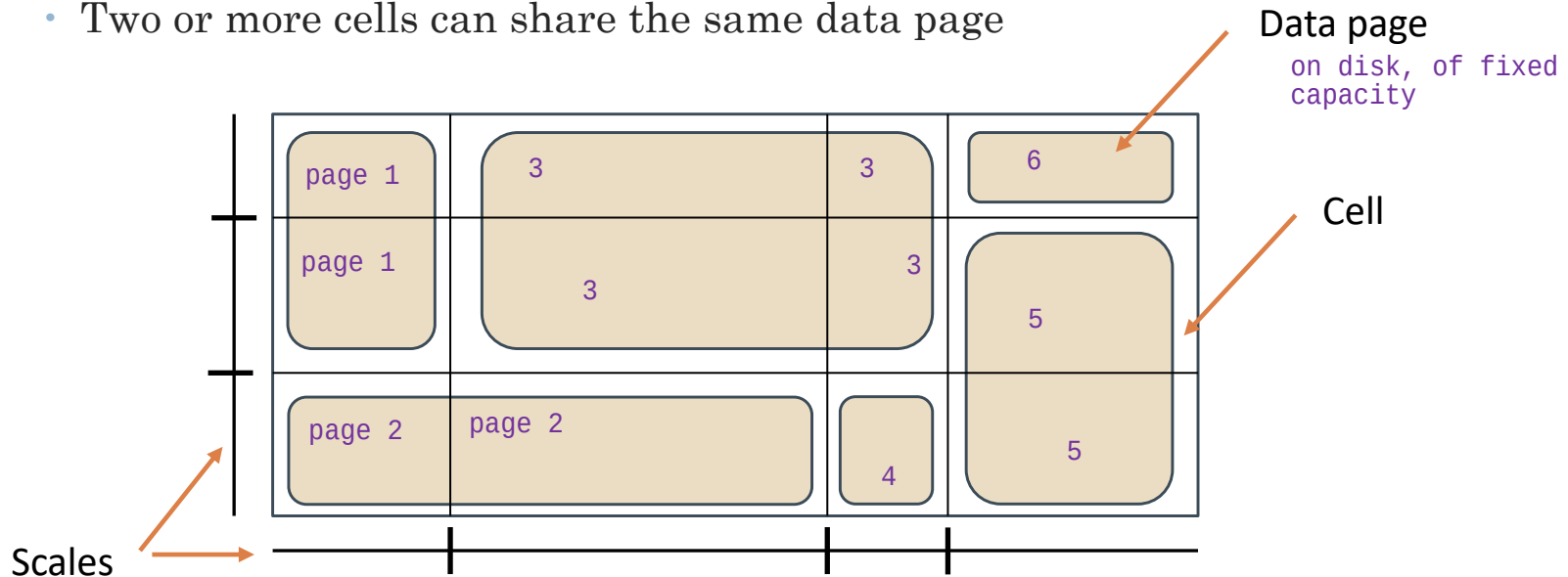
Zineddine Kouahla et al. A Survey on Big IoT Data Indexing: Potential Solutions, Recent Advancements, and Open Issues. Future Internet 14(1): 19 (2022)

Spatial indices: classification

- Type of objects
 - For points (records don't have a spatial extension)
 - For lines/regions
- Type of subdivision
 - **Space partitioning**: splits are performed according to global considerations
 - Good for uniform distributions, easier to implement
 - **Data partitioning**: splits are performed according to local considerations, à la B⁺-tree
 - Good for **arbitrary distributions**, more complex to implement
- Type of organization
 - Tree-/hash-based
- Common approach:
 - The space is organized in regions (sometimes called *cells*)
 - Each cell corresponds (not always 1-1) to a data page

Grid File

- The Grid File organizes the space into a grid, where each dimension is partitioned into arbitrarily sized intervals
 - **d scales** (1-dim arrays) are required, containing values used as separators for each dimension
- Each grid cell points to a data page on disk
 - Two or more cells can share the same data page

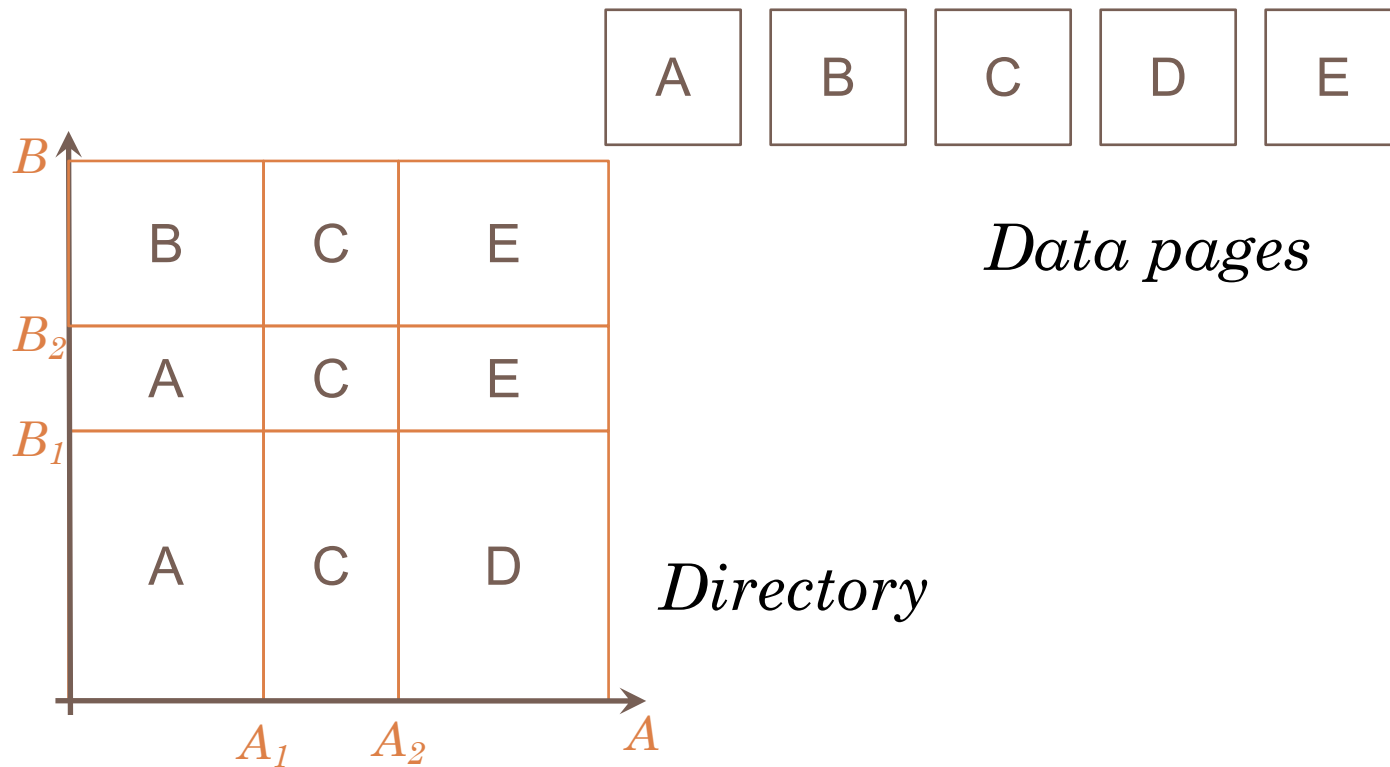


Jürg Nievergelt, Hans Hinterberger, Kenneth C. Sevcik.

The Grid File: An Adaptable, Symmetric Multikey File Structure. ACM TODS 9(1): 38-71 (1984)

Grid File: data page overflows

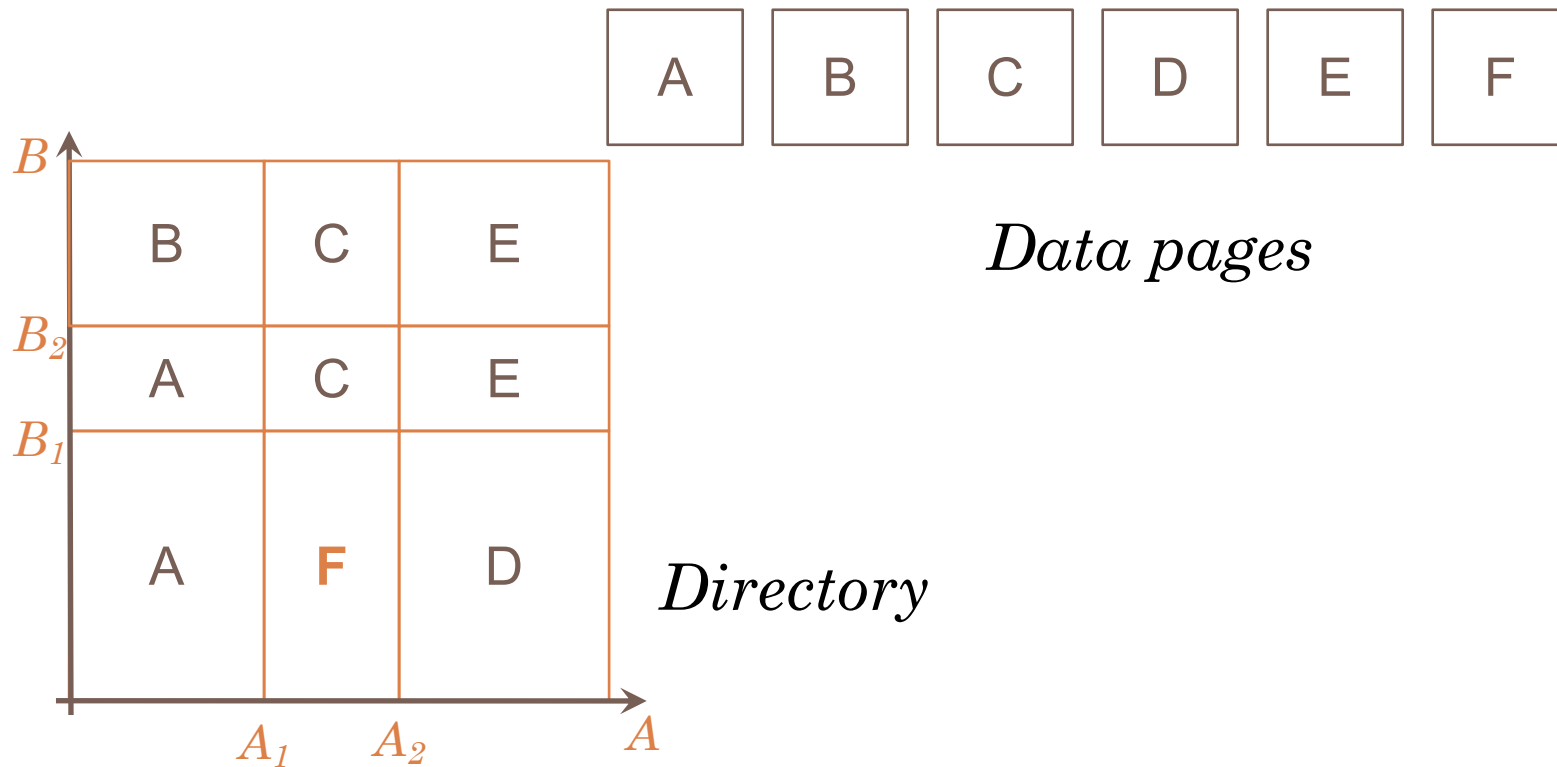
- When a data page overflows, it is split you need to allocate more space on disk but ALSO to restructure the index
- For the directory, we can have one of two cases
 - If the block was referenced by **two (or more)** cells, we only update pointers
 - **Otherwise**, we add a separator to the directory



Grid File: split (i)

- First case: C overflows and is split into C and F
- It is sufficient to update the pointer of the cell

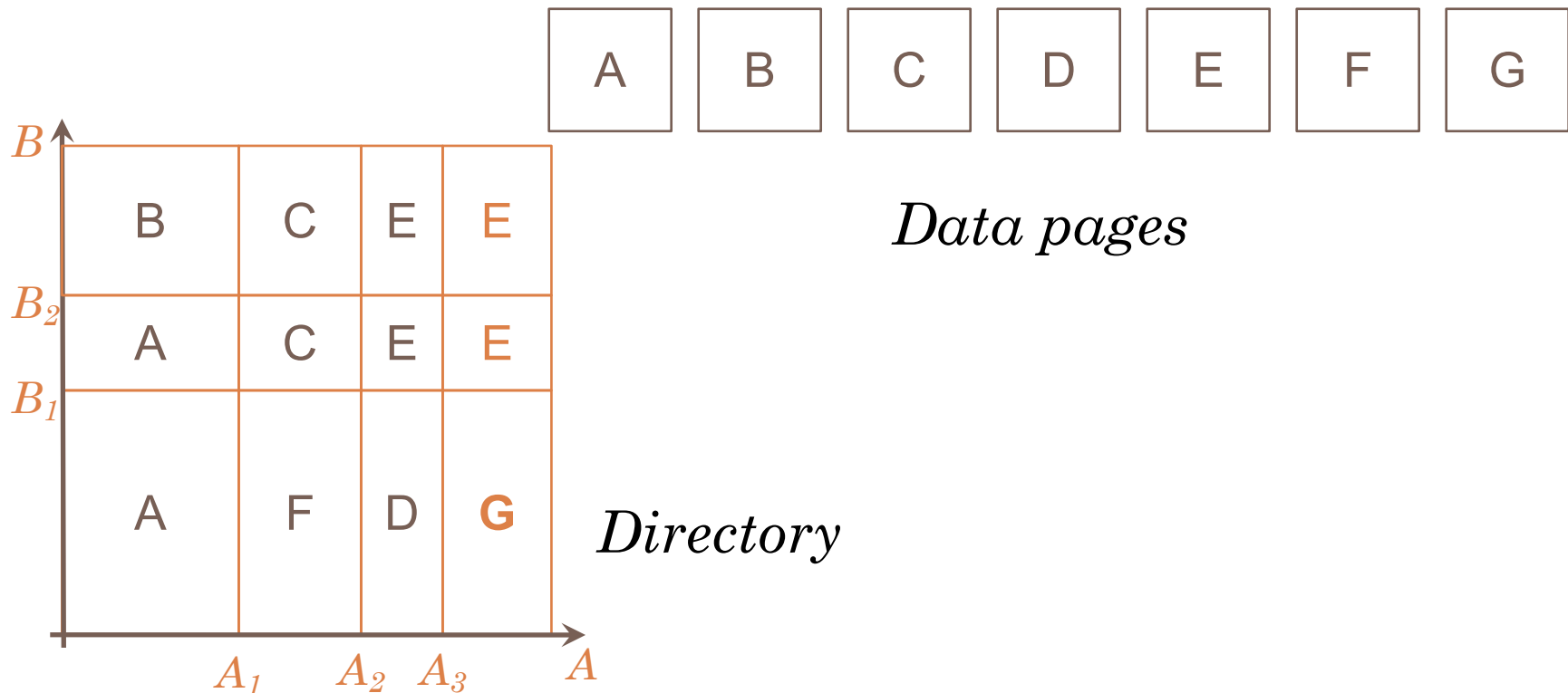
F is the new page allocated on disk, just re-wire the pointers for each cell and move the data in $[A_1, A_2] \times [0, B_1]$ over to F



Grid File: split (ii)

- Second case: **D overflows and is split into D and G**
- We have to augment the directory using an additional separator, e.g., for coordinate A

this is the case when
D was a single region
and we need to split it
--> add separator to one of
the dimensions



- In case of non-uniform distributions, storing n points could require a number of cells which grows like $O(n^d)$

In general, it behaves poorly with correlated data

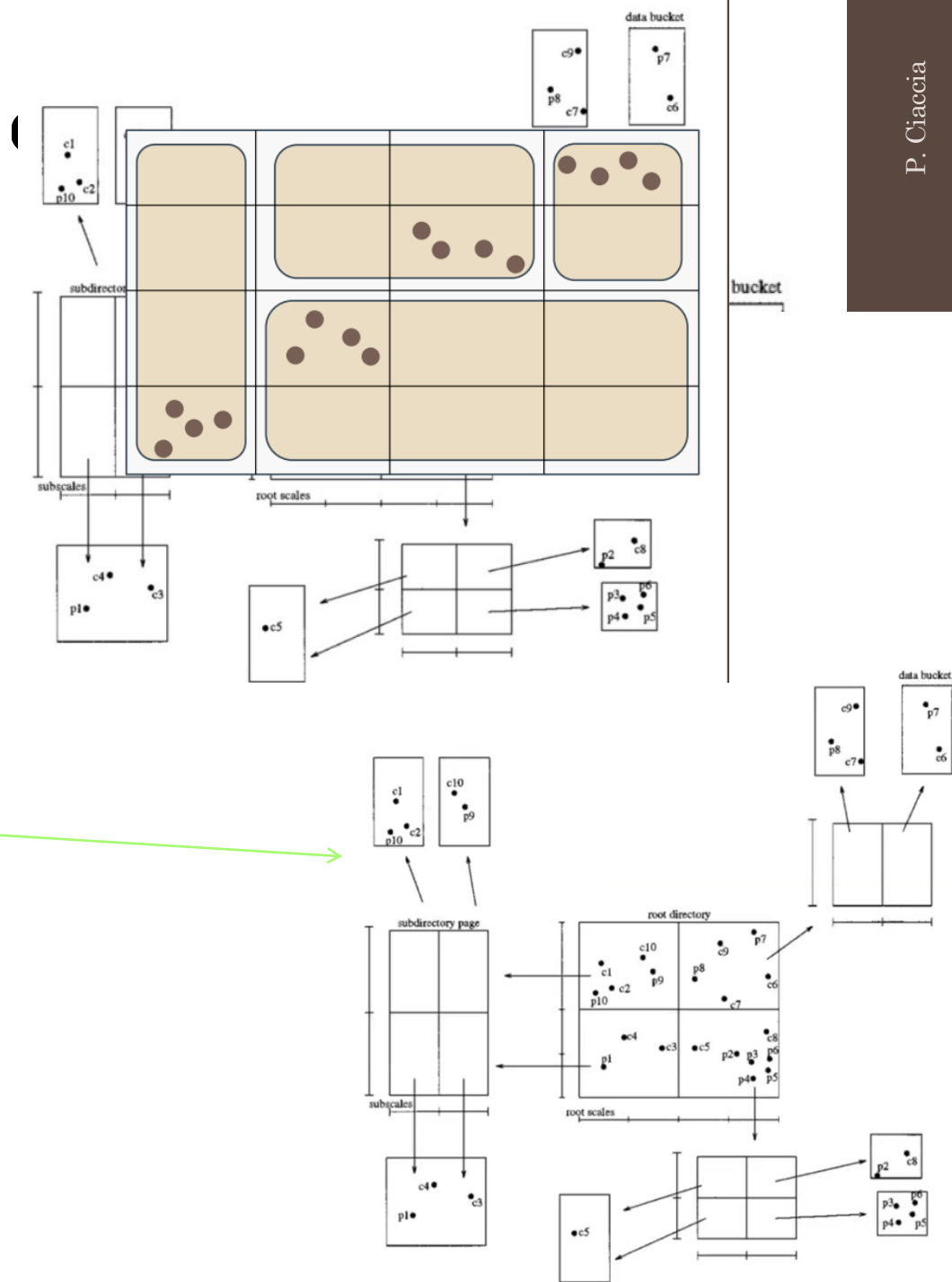
- On the other hand, the regular structure of space partitioning greatly simplifies the resolution of window queries

- Main problem: directory management

Usually, scales are stored in main memory

In (quasi-)static cases, the directory can be stored on disk as a multi-dimensional array

In dynamic cases, it is necessary to paginate the directory, leading to multi-level grid files



R-tree

node = page = block
= minimal unit of storage in disk

B+ trees nested 1-dimensional intervals
in a hierarchical structure. The idea
here is the same, only for multi-dimensions.

- **Balanced** and **paginated** tree structure,
based on the hierarchical nesting of **overlapping regions**
- Each node corresponds to a **rectangular region**,
defined as the **MBB** (Minimum Bounding Box) of its children's regions
- Storage utilization for each node varies from 100%
to a minimum value ($\leq 50\%$), which is a design parameter
 - $< 50\%$: more flexibility when managing splits in B+ it's roughly almost always 50%
because it's easier to split intervals
in half for 1 dimensions
- Management mechanisms similar to those of the B⁺-tree,
with the main difference that insertion of an object and splits can be
managed according to different policies

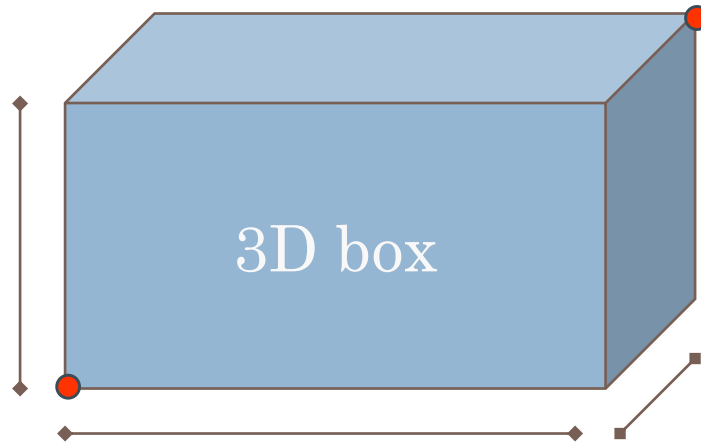
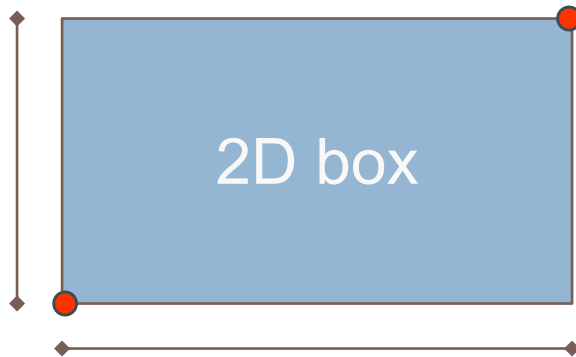
Antonin Guttman.

R-Trees: A Dynamic Index Structure for Spatial Searching. SIGMOD Conference 1984: 47-57

R-tree: concept of MBB

- **MBB = Minimum Bounding Box**

- The smallest rectangle, with sides parallel to coordinate axes, containing all children regions
- It is the product of d intervals



- A MBB is univocally defined by the (2d) coordinates of two any opposite vertices (usually: lower-left & upper-right)
- The MBB of m points can be computed in $O(md)$ time
 - It is sufficient to find the minimum and maximum value for each coordinate

R-tree: comparison with B⁺-tree

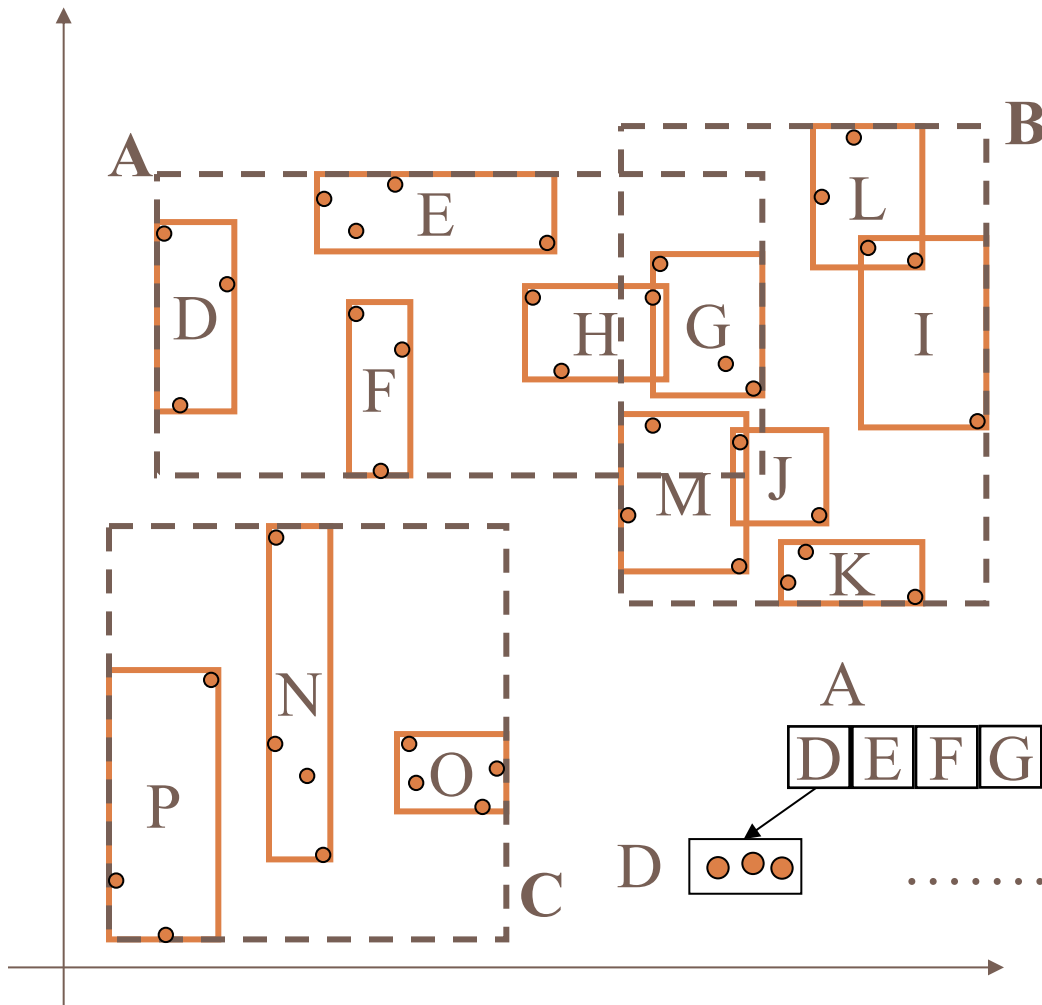
B⁺-tree

- Balanced and paginated tree
- Data are stored in leaves
- Leaves are kept sorted and linked
- Data are organized into 1D intervals
 - Intervals do not overlap
- This principle is recursively applied towards the root
- Point search follows a single path from root to a single leaf

R-tree

- Balanced and paginated tree
- Data are stored in leaves
- No data order exist
- Data are organized into d-dim intervals (MBB)
 - Intervals do overlap (characteristic of $d > 1$ spaces)
- This principle is recursively applied towards the root
- Point search could follow multiple paths from root to multiple leaves

R-tree: organization



- Recursive bottom-up aggregation of objects based on MBB's
- Regions can overlap
- Each node can contain up to C_{max} entries, but not less than $C_{min} \leq 0.5 * C_{max}$
 - The root makes an exception

distinct points can store overlapping regions... redundancy of data?

R-tree: characteristics

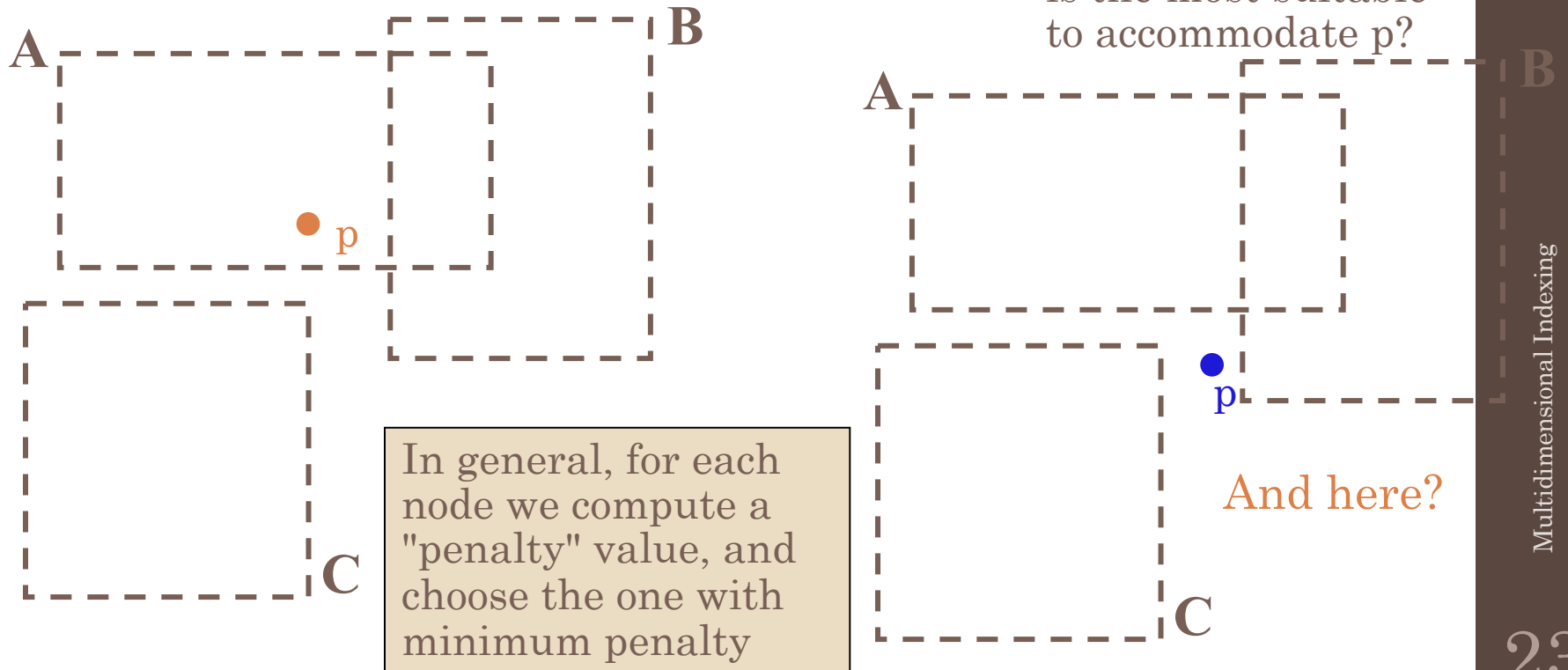
- Leaf nodes
 - Contain entries with the form (key, RID), where key stores the coordinates of the record referenced by RID
 - Actually, R-tree could also store d-dim objects with a spatial extension, with key = MBB(object)
- Internal nodes
 - Contain entries of the form (MBB, PID), where MBB stores the coordinates of the MBB containing all entries reachable from the node referenced by PID
- Overall, each node contains entries with the form (key, ptr), where key is a “spatial” value

R-tree: insertion of a new object

in B+ trees it's easy because the path is unique, here we have more roads to follow

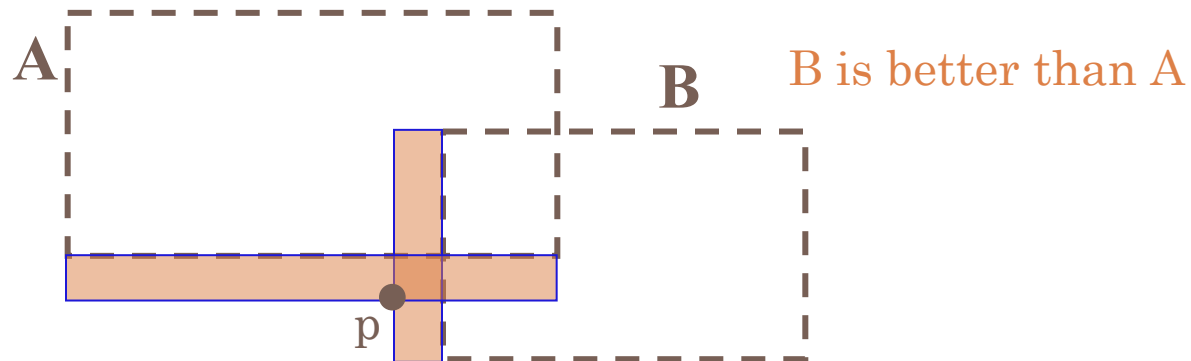
- We start from the root and move down the tree one step at a time, trying to find a “nice place” where to accommodate the new object p
 - For simplicity, we assume that indexed objects are points, similar arguments apply if we index (hyper-)rectangles (MBB's)

- At each step we have a same question to answer: Which child node is the most suitable to accommodate p ?



R-tree: the Penalty method

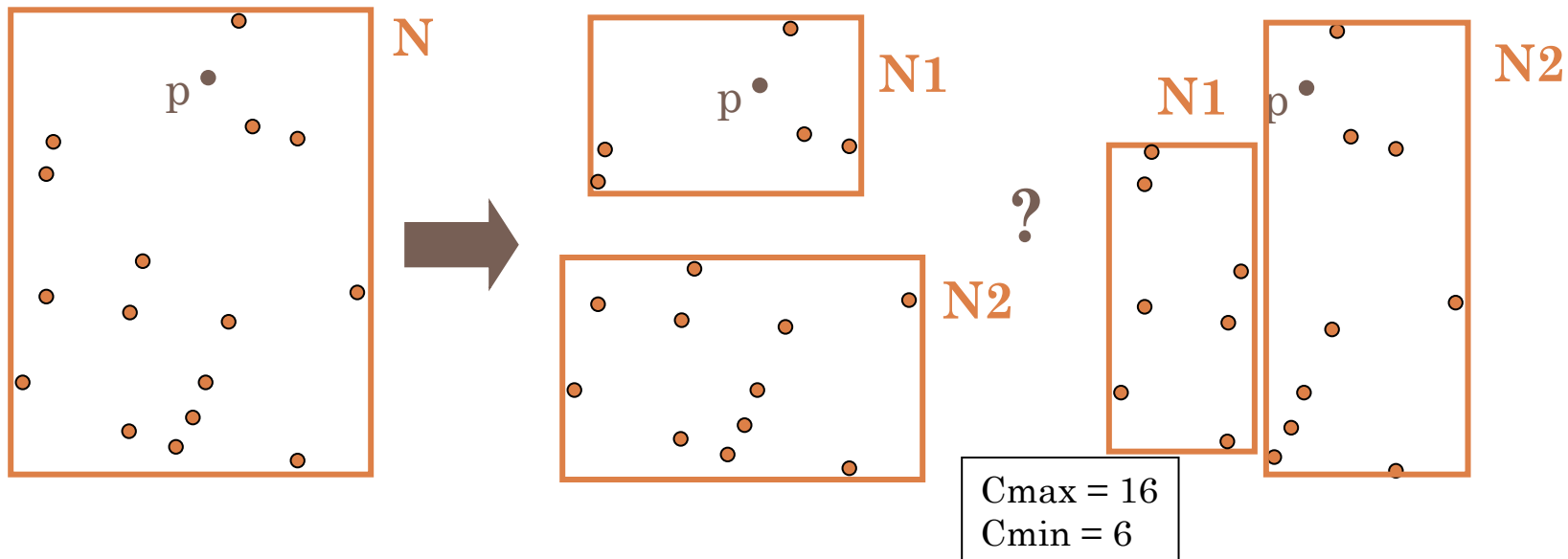
- If point p is inside the region of a node, then the Penalty is 0
- Otherwise, Penalty can be computed as the increment of volume (area) of the MBB
 - For leaf nodes, the R*-tree variant considers the increment of overlap with the other regions
- Both criteria aim to obtain trees with better performance:
 - **Large volume:** the chance of visiting the node during a query **increases**
 - **Large overlap:** the number of nodes visited during a query **increases**



Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, Bernhard Seeger.
The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. SIGMOD Conference 1990: 322-331

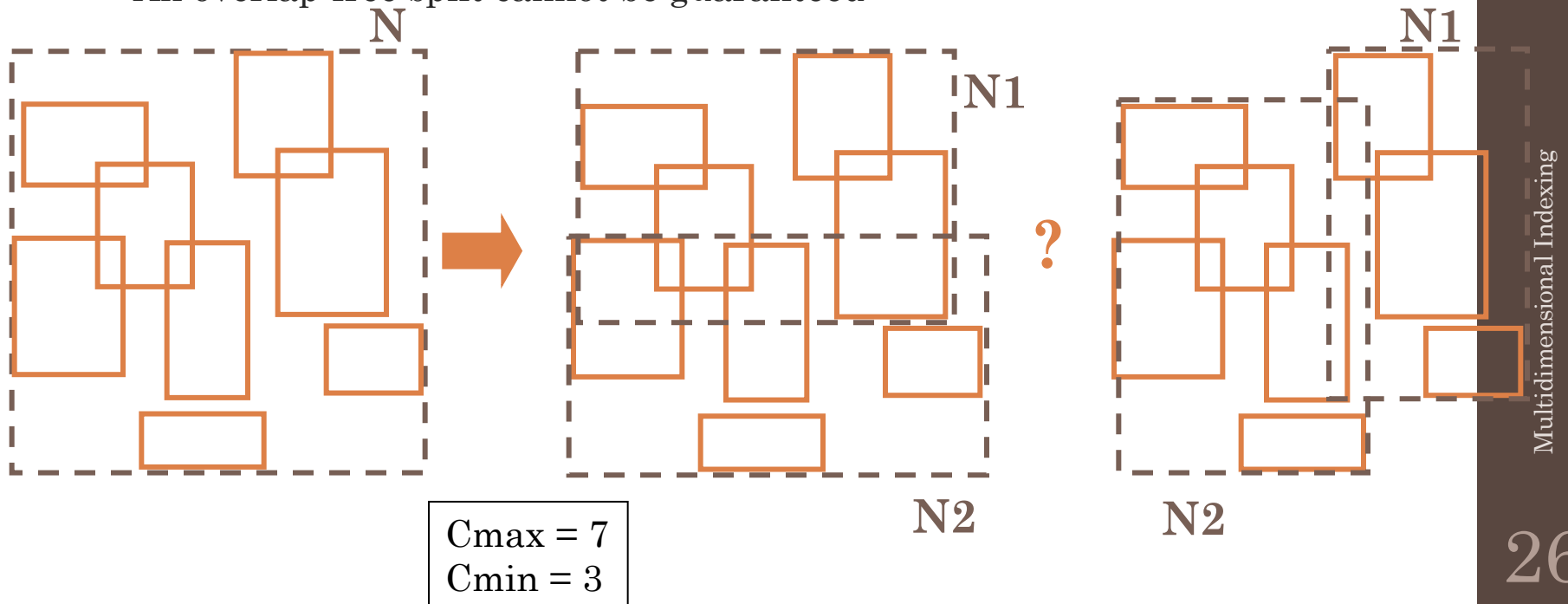
R-tree: splitting a leaf node

- When p has to be inserted into a leaf node N that already contains $C_{\max}$ entries, an overflow occurs, and N has to be split
- For leaf nodes whose entries are points the solution aims to split the set of $C_{\max}+1$ points into 2 subsets, each with at least $C_{\min}$ and at most $C_{\max}$ points
- Among the several possibilities, one could consider the choice that leads to have a minimum overall volume
 - NP-Hard problem, thus heuristics have to be applied



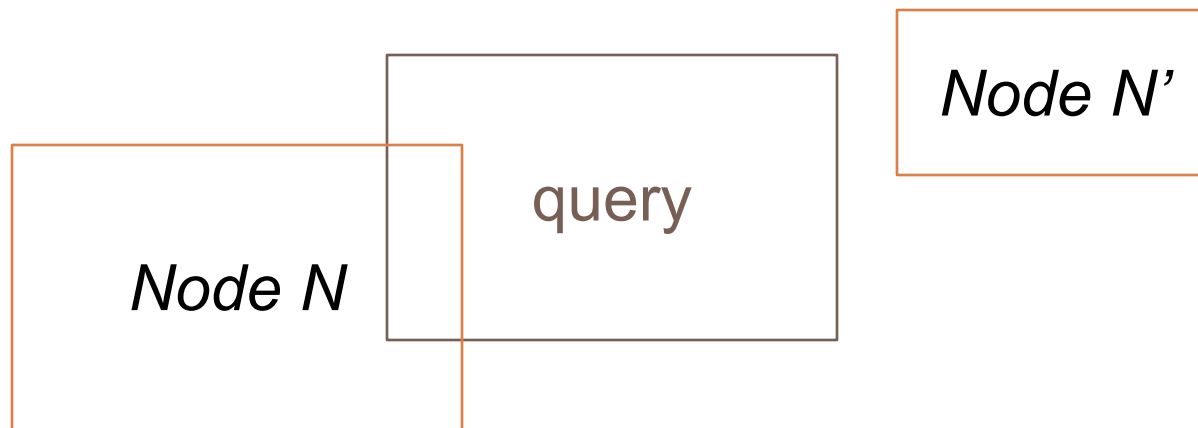
R-tree: splitting an internal node

- As in B+-trees, splits propagate upward and can recursively trigger splits at higher levels of the tree
- How to split a set of $C_{\max}+1$ (hyper-)rectangles?
 - The R-tree just aims to minimize the sum of resulting volumes
 - The R*-tree implements a more sophisticated criterion, which takes into account the volume, overlap, and surface of the resulting regions
 - An overlap-free split cannot be guaranteed

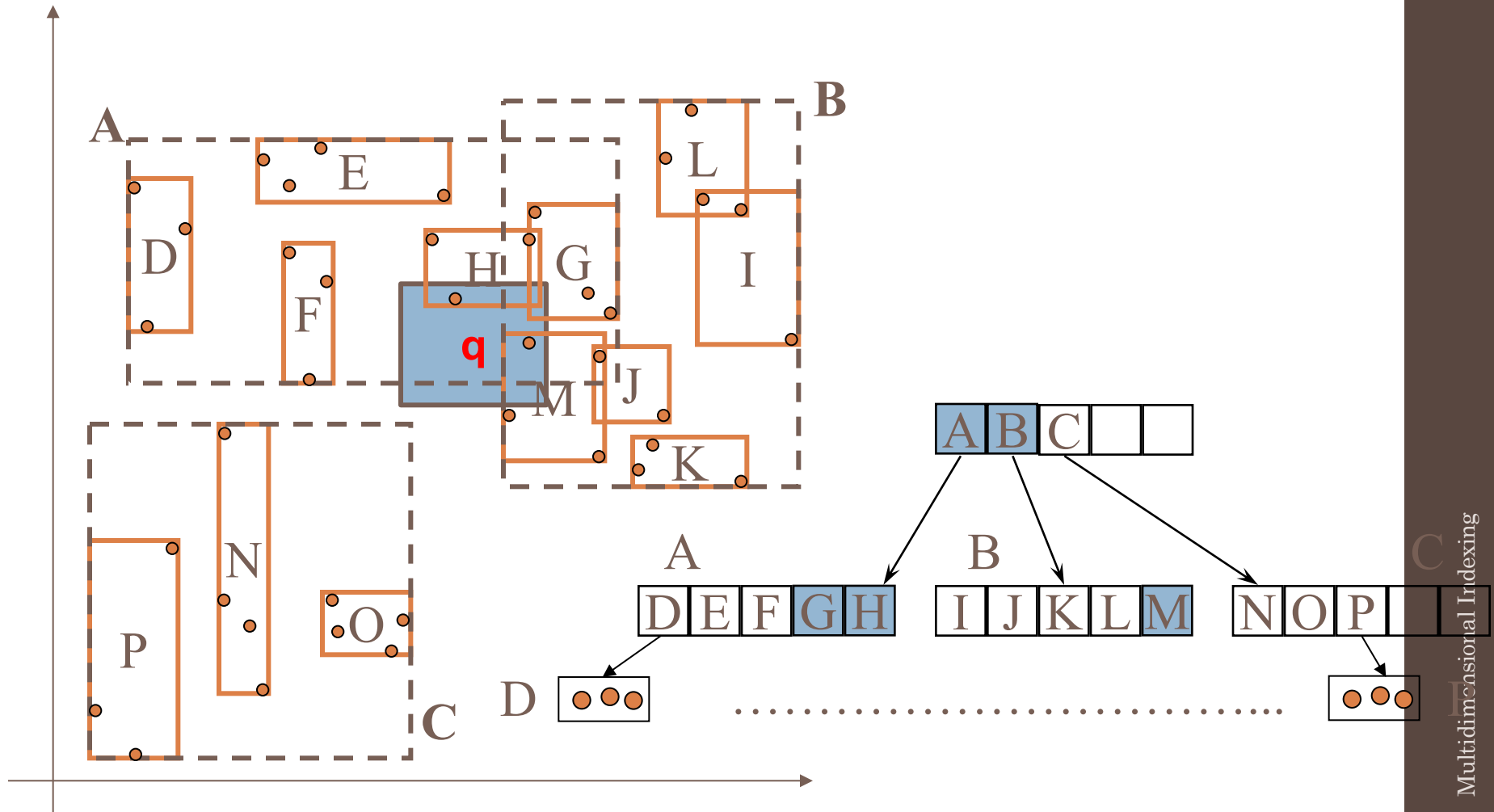


R-tree: search (window query)

- We have to retrieve all points included into a product of d intervals (that is, a box)
- Such points could only be found in nodes whose MBB overlaps with the query region
- E.g.: node N' cannot contain records satisfying the query

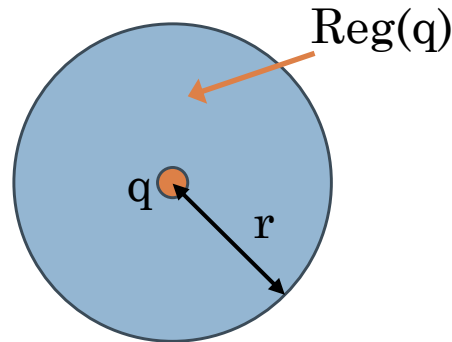


R-tree: window query example



Search: range query

- We have 3 parameters:
 - The query point q
 - The search radius r
 - The distance function D

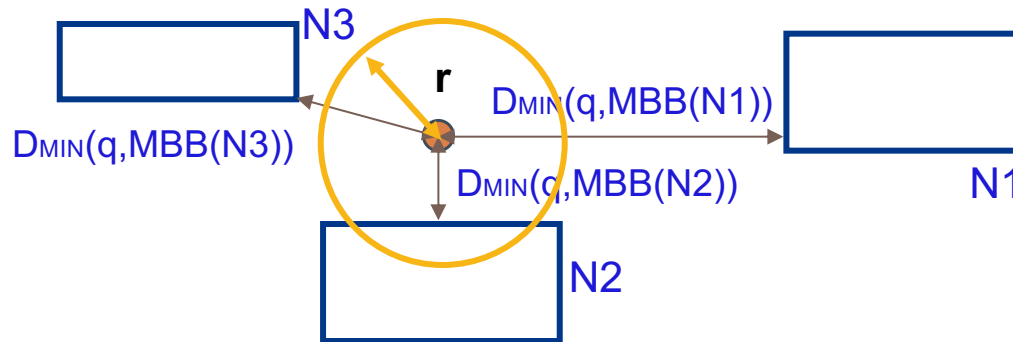


D : Euclidean distance

- The **query region** is defined as $Reg(q) = \{p: p \in \mathbb{R}^d, D(p,q) \leq r\}$
 - For simplicity, both D and r are understood in the notation $Reg(q)$
- The algorithm for processing a range query is extremely simple:
 - Start from the root; for each node N , check if $MBB(N)$ intersects $Reg(q)$
 - On leaf nodes, for each point p check if $D(p,q) \leq r$.

The D_{MIN} lower bound

- For a node N , let $D_{\text{MIN}}(\mathbf{q}, N) = \inf_p \{D(\mathbf{q}, p) \mid p \in \text{MBB}(N)\}$ be the minimum possible distance between $\mathbf{q}$ and a point reachable from N



Basic fact:

$$\text{Reg}(\mathbf{q}) \cap \text{MBB}(N) \neq \emptyset$$



$$D_{\text{MIN}}(\mathbf{q}, N) \leq r$$

Common distance functions (i)

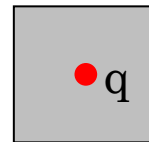
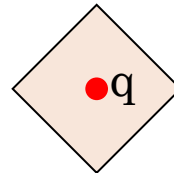
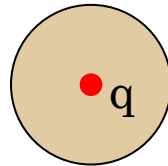
- The most commonly used distance functions are Lp-norms:

$$L_p(t, q) = \left(\sum_{i=1}^d |t_i - q_i|^p \right)^{1/p}$$

$$L_2(t, q) = \sqrt{\sum_{i=1}^d |t_i - q_i|^2} \quad \text{Euclidean}$$

$$L_1(t, q) = \sum_{i=1}^d |t_i - q_i| \quad \text{Manhattan}$$

$$L_\infty(t, q) = \max_i \{|t_i - q_i|\} \quad \text{Chebyshev (max)}$$



Common distance functions (ii)

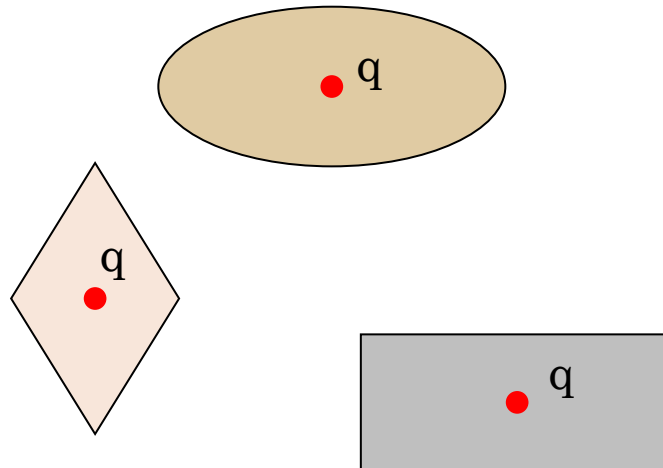
- ... and their weighted versions:

$$L_p(t, q) = \left(\sum_{i=1}^d w_i |t_i - q_i|^p \right)^{1/p}$$

$$L_2(t, q; W) = \sqrt{\sum_{i=1}^d w_i |t_i - q_i|^2}$$

$$L_1(t, q; W) = \sum_{i=1}^d w_i |t_i - q_i|$$

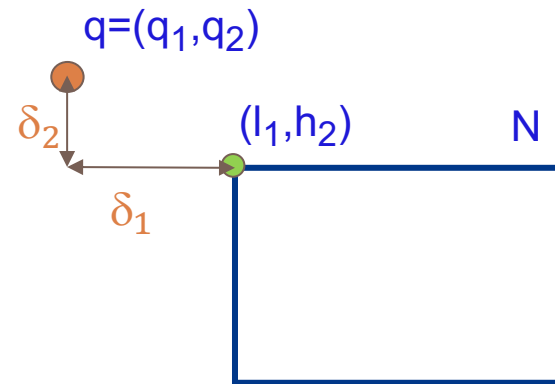
$$L_\infty(t, q; W) = \max_i \{w_i |t_i - q_i|\}$$



Computing D_{MIN} for weighted Lp norms

- Computing D_{MIN} for a weighted Lp norm has complexity $O(d)$
- Let $\text{MBB}(N) = [l_1, h_1] \times \dots \times [l_d, h_d]$
- For the i-th coordinate let us define the "offset" of the query point $q = (q_1, \dots, q_d)$ with respect to $\text{MBB}(N)$ as:

$$\delta_i = \begin{cases} q_i - h_i & \text{if } q_i \geq h_i \\ l_i - q_i & \text{if } l_i \geq q_i \\ 0 & \text{otherwise} \end{cases}$$



- Then, D_{MIN} is computed as:

$$L_{p, \text{MIN}}(q, N; W) = \left(\sum_{i=1}^d w_i \delta_i^p \right)^{1/p}$$

Search: k-NN query

- The **kNNOptimal** algorithm can solve **k-NN queries with an R-tree in an I/O-optimal way** reads min num nodes possible from disk.
 - The algorithm also applies to all index structures such that the region of any child node is included in the one of its parent (e.g., the M-tree) containment property, this is enough for the algorithm to work!
- Basic case: $k=1$
- Let t_{NN} be the 1st nearest neighbor of q ($1\text{-NN} = \text{NN}$), and let $r_{NN} = D(q, t_{NN})$ be its distance from q
 - Clearly, r_{NN} is only known when the algorithm terminates

Theorem:

Any correct algorithm for 1-NN queries must visit at least all the nodes N whose D_{MIN} is strictly less than r_{NN} , i.e., $D_{MIN}(q, N) < r_{NN}$

Proof: If an algorithm A stops by reporting as NN of q a point t , and A does not read a node N such that $D_{MIN}(q, N) < D(q, t)$, then N might lead to a point t' with $D(q, t') < D(q, t)$, thus contradicting the hypothesis that t is the NN of q .

Stefan Berchtold, Christian Böhm, Daniel A. Keim, Hans-Peter Kriegel.

A Cost Model For Nearest Neighbor Search in High-Dimensional Data Space. PODS 1997: 78-86

The logic of the kNNOptimal Algorithm

- kNNOptimal uses a **priority queue** PQ, whose elements are pairs $[\text{ptr}(N), D_{\text{MIN}}(q, N)]$
- **PQ is ordered by increasing values of $D_{\text{MIN}}(q, N)$**
 - DEQUEUE extracts from PQ the pair with minimal D_{MIN}
 - ENQUEUE($[\text{ptr}(N), D_{\text{MIN}}(q, \text{MBB}(N))]$) performs an ordered insertion of the pair in the queue
- Pruning of the nodes is based on the following observation:

If we have found a point t with $D(q, t) = r$,
then all nodes N with $D_{\text{MIN}}(q, N) \geq r$ can be excluded from the search

- In the description of the algorithm, the pruning of pairs of PQ based on the above criterion is concisely denoted as UPDATE(PQ)
 - Alternatively: leave (the pairs corresponding to) such nodes in the queue and modify the stopping criterion, i.e., stop when the result cannot be improved anymore

The kNNOptimal Algorithm (k=1)

Input: query point q , index tree with root node RN

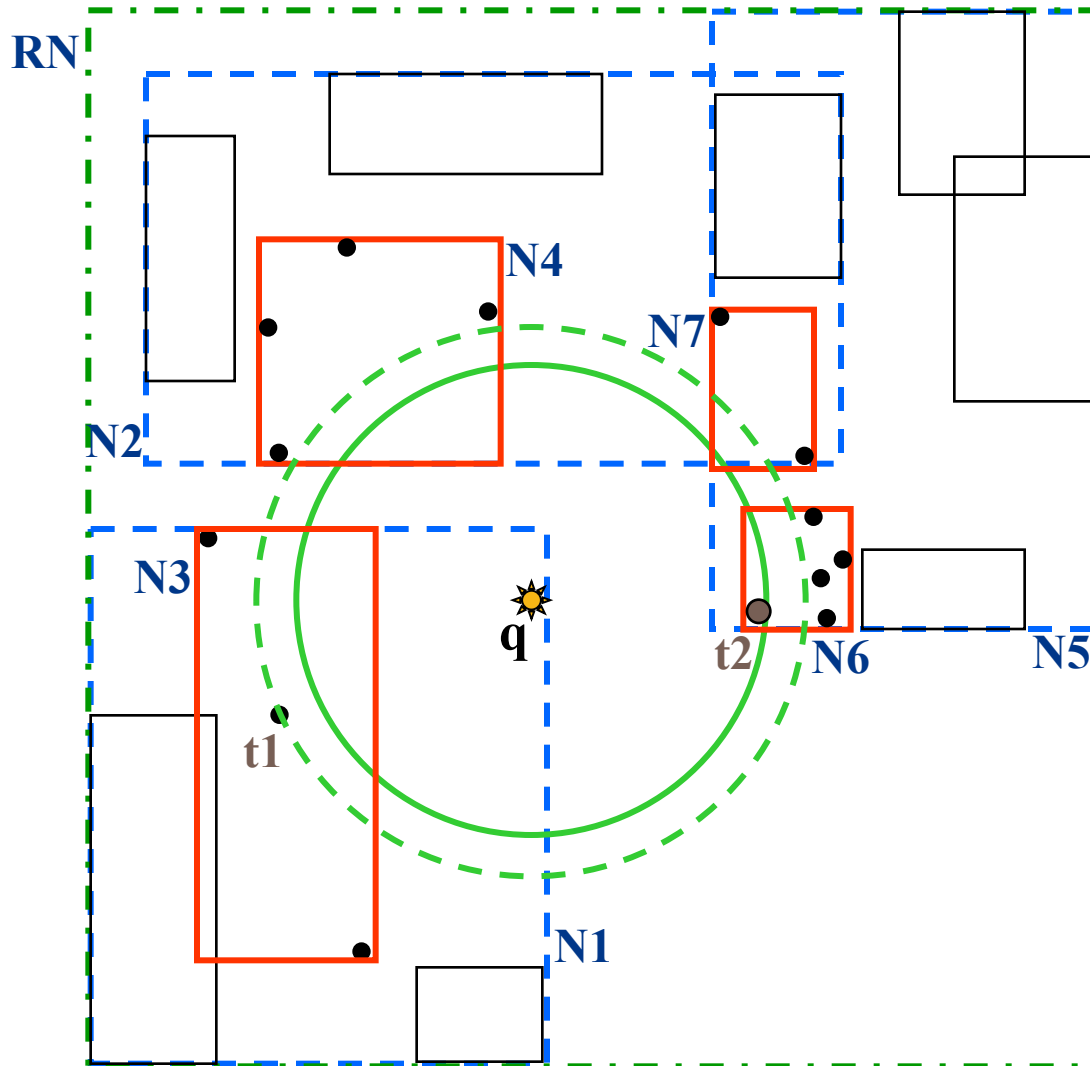
Output: t_{NN} , the nearest neighbor of q , and $r_{NN} = D(q, t_{NN})$

```

NearestNeighborSearch( $q$ ,  $R\_tree$ ):
     $PQ = \{ [ptr(get\_root(R\_tree)), 0] \}$ 
     $t_{NN} = null$ 
     $r_{NN} = \infty$ 
    while is_nonempty( $PQ$ ) { // go on until the PQ is entirely consumed
         $[ptr(N), DMIN(q, N)] = DEQUEUE(PQ)$  // this consumes the queue
         $N = read(ptr(N))$ 

        if isLeaf( $N$ ) {
            foreach  $t$  in get_points( $N$ )
                if  $D(q, t) < r_{NN}$  {
                     $t_{NN} = t$ 
                     $r_{NN} = D(q, t)$ 
                    PRUNE_WORSE_NODES( $PQ, r_{NN}$ ) // "UPDATE" in prev. slides
                }
        } else {
            foreach  $N_c$  in get_children( $N$ )
                if  $DMIN(q, N_c) < r_{NN}$ 
                    ENQUEUE( $PQ, [ptr(N_c), DMIN(q, N_c)]$ )
        }
    }
    return ( $t_{NN}, r_{NN}$ )
  
```

kNNOptimal in action



- Nodes are numbered following the order in which they are accessed
- Objects are numbered as they are found to improve (reduce) the search radius
- The accessed leaf nodes are shown in red

Optimality of kNNOptimal

- The kNNOptimal algorithm is clearly correct
- To show that it is also I/O-optimal, that is, it reads the minimum number of nodes, it is sufficient to prove the following

Theorem:

The kNNOptimal algorithm for 1-NN queries never reads a node N whose D_{MIN} is strictly larger than r_{NN} , i.e., $D_{\text{MIN}}(q, N) > r_{\text{NN}}$

Proof:

- Node N is read only if, at some execution step, it becomes the 1st element in PQ
- Let N_1 be the node containing t_{NN} , N_2 its parent node, N_3 the parent node of N_2 , and so on, up to $N_h = \text{RN}$ (h = height of the tree)
- Now observe that, by definition of D_{MIN} , it is:

$$r_{\text{NN}} \geq D_{\text{MIN}}(q, N_1) \geq D_{\text{MIN}}(q, N_2) \geq \dots \geq D_{\text{MIN}}(q, N_h)$$

- At each time step before we find t_{NN} , one (and only one) of the nodes $N_1, N_2, \dots, N_h$ is in the priority queue
- It follows that N can never become the 1st element of PQ ■

The general case ($k \geq 1$)

- The algorithm is easily extended to the case $k \geq 1$ by using:
 - a data structure (Res), where we maintain the k closest objects found so far, together with their distances from q
 - as "current search radius" the distance, $r_{k\text{-NN}}$, of the current k -th NN of q , that is, the k -th element of Res

Res

ObjectID	distance
t15	4
t24	8
t18	9
t4	12
t2	15

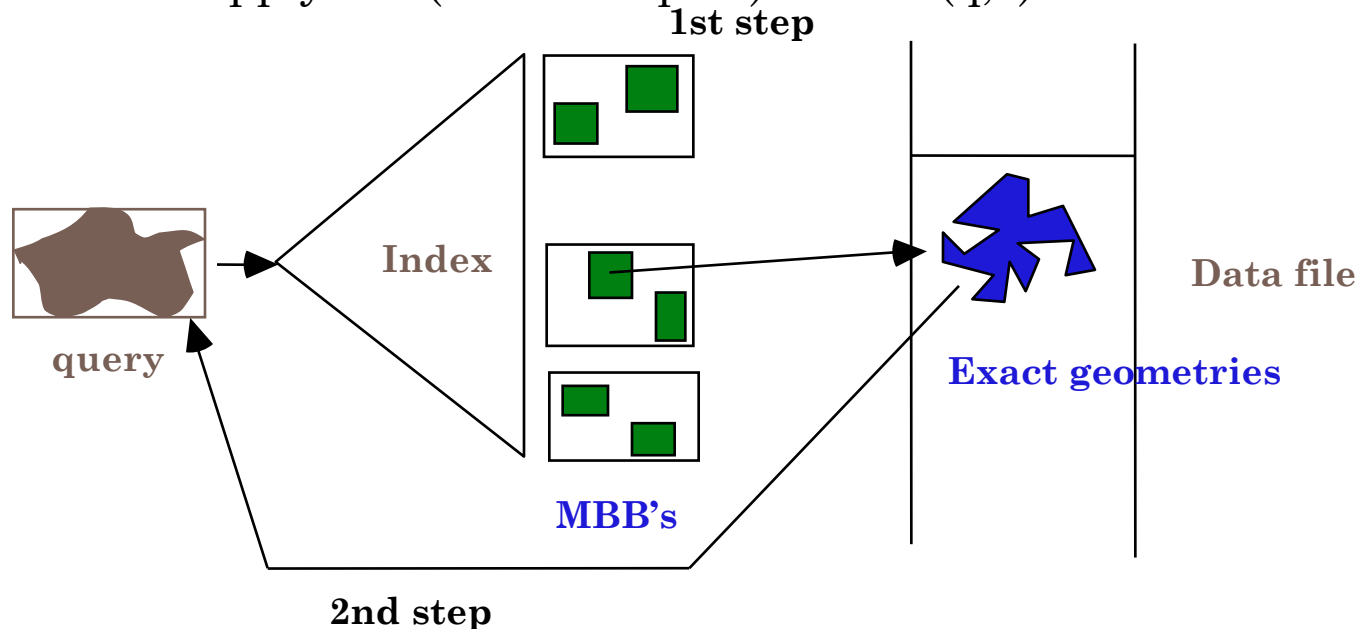
$k = 5$

No node with $D_{\text{MIN}} \geq 15$
needs to be read

- The rest of the algorithm remains unchanged

R-tree: indexing regions

- Leaves can store MBB's of regions, rather than points
- Assume you want to determine all the data regions o that are at distance $\leq r$ from the query region, i.e., $L2(q,o) \leq r$
cheap, not precise step.
- [Filter] In the 1st step, we use the index and retrieve all the MBB's such that $L2(MBB(q), MBB(o)) \leq r$
- [& Refine] In the 2nd step, we access the actual geometries of the objects and apply the (more complex) test $L2(q,o) \leq r$



Beyond vector spaces

- Vector spaces, equipped with some (weighted) Lp-norm, are not general enough to deal with the whole variety of feature types and distance functions needed for MM data

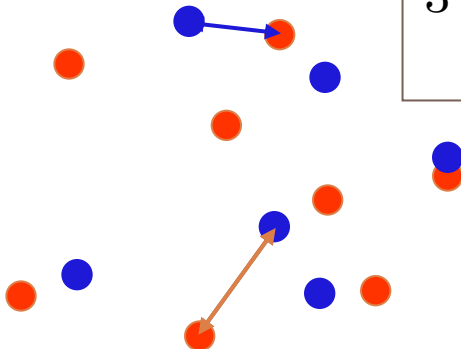
Example 1:

given 2 sets of points S1 and S2, their **Hausdorff distance** is defined as follows:

- 1 $\forall$ (red) point of S1 find the closest (blue) point in S2
Let $h(S1, S2)$ be the maximum of such distances
- 2 $\forall$ (blue) point in S2 find the closest (red) point in S1
Let $h(S2, S1)$ be the maximum of such distances
- 3 Let $D_{Haus}(S1, S2) = \max\{ h(S1, S2), h(S2, S1) \}$

Used for matching shapes

$h(S1, S2)$ = "how well is S1 covered by S2?" – for every red point, how far is the nearest blue point?
 $h(S2, S1)$ = "how well is S2 covered by S1?" – for every blue point, how far is the nearest red point?



Example 2: set similarity

- We have **logs of WWW accesses**, where each log entry has a format like:

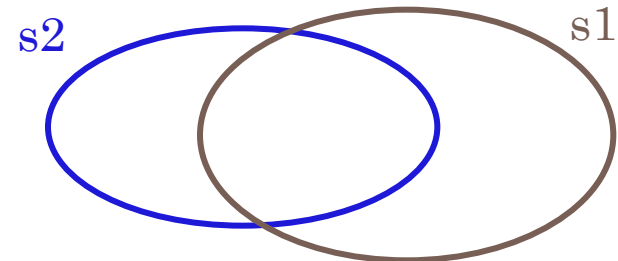
```
www-db.deis.unibo.it pciaccia -
[11/Jan/1999:10:41:37 +0100]
"GET /~mpatella/ HTTP/1.0" 200 1573
```

- Log entries are grouped into **sessions (= sets of visited pages)**:

$s = \langle \text{ip_address}, \text{user_id}, [\text{url}_1, \dots, \text{url}_k] \rangle$

and we want to compare “**similar sessions**” (i.e., similar sets), using a (normalized) symmetric difference:

$$D_{\text{setdiff}}(s1, s2) = \frac{|s1 - s2| + |s2 - s1|}{|s1| + |s2|}$$



Example 3: edit distance

- A common distance measure for strings, defined as the minimum number of characters that have to be inserted, deleted, or substituted so as to transform a string s_1 into another string s_2

$$D_{\text{edit}}(\text{'ball'}, \text{'bull'}) = 1 \quad D_{\text{edit}}(\text{'balls'}, \text{'bell'}) = 2 \quad D_{\text{edit}}(\text{'rather'}, \text{'alter'}) = 3$$

- The edit distance is also used in **genomic DB's** to compare **DNA sequences**.
- Each DNA sequence is a string over the 4-letters alphabet of bases:

$$D_{\text{edit}}(\text{'gatctggtgg'}, \text{'agcaaatacag'}) = 7$$

a: adenine
 c: cytosine
 g: guanine
 t: thymine

g	a	t	c	t	g	g	t	g	-	g
1	=	2	=	3	4	5	=	6	7	=
-	a	g	c	a	a	a	t	c	a	g

Computing the Edit Distance

- The cost matrix (on the left) is used to incrementally build the new matrix D_{edit} , whose elements are recursively defined as:

$$D_{\text{edit};i,j} = \text{cost}_{i,j} + \min\{D_{\text{edit};i-1,j}, D_{\text{edit};i,j-1}, D_{\text{edit};i-1,j-1}\}$$

cost

r	1	1	1	1	1	0
e	1	1	1	1	0	1
h	1	1	1	1	1	1
t	1	1	1	0	1	1
a	1	0	1	1	1	1
r	1	1	1	1	1	0
	0	1	1	1	1	1
		a	l	t	e	r

s2

s1

D_{edit}

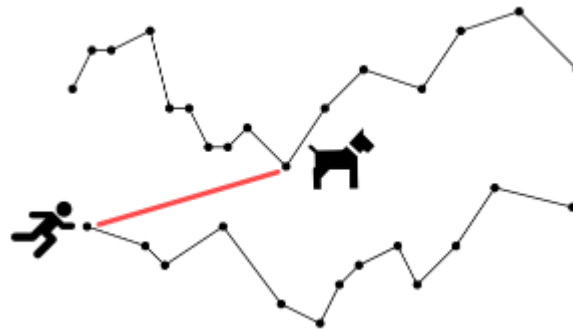
r	6	5	5	5	4	3
e	5	4	4	4	3	4
h	4	3	3	3	3	4
t	3	2	3	2	3	4
a	2	1	2	3	4	5
r	1	1	2	3	4	4
	0	1	2	3	4	5
		a	l	t	e	r

s2

s1

Example 4: Fréchet distance

- The Fréchet (aka the dog-keeper) distance is a measure of similarity **between curves** (more effective than Hausdorff distance)
 - speech recognition, signature and handwriting recognition, matching of time series, moving objects analysis, etc.
- Imagine a person traversing a finite path while walking her dog on a leash, with the dog traversing a separate finite path
 - Backtracking is not allowed
- Which is the length of the shortest leash sufficient for both to completely traverse their paths?



Metric spaces

- A metric space $M = (U, d)$ is a pair, where:
U is a domain (“universe”) of values, and
D is a distance function that, $\forall x, y, z \in U$, satisfies the metric axioms:

$$D(x, y) \geq 0, D(x, y) = 0 \Leftrightarrow x = y \quad (\text{positivity})$$

$$D(x, y) = D(y, x) \quad (\text{symmetry})$$

$$D(x, y) \leq D(x, z) + D(z, y) \quad (\text{triangle inequality})$$

- All the distance functions seen in the previous examples are metrics, and so are the (weighted) L_p -norms
- Metric indexes only use the metric axioms to organize objects, and exploit the triangle inequality to prune the search space
- Besides I/O costs, now also CPU costs (distance computations) can heavily influence performance

can we index metric spaces, dropping coordinates for points and using only distances? --> cpu costs for the distance computation

Principles of metric indexing (1)

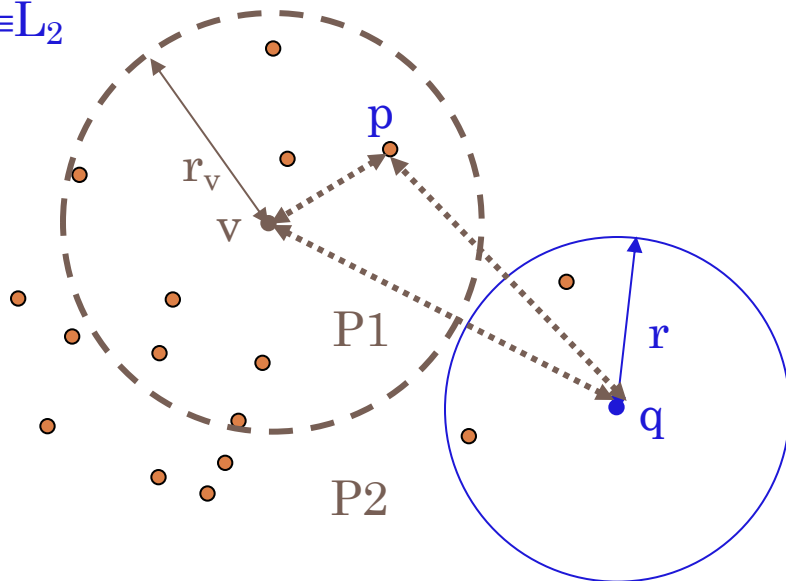
- Given a “metric dataset” $P \subseteq U$, one of the two following principles can be applied to partition it into two subsets

Ball decomposition: take a point v (“**vantage point**”), compute the distances of all other points p w.r.t. v , $D(p,v)$, and define

$$P1 = \{p : D(p,v) \leq r_v\}$$

$$P2 = \{p : D(p,v) > r_v\}$$

$D \equiv L_2$



so basically we index the dataset a priori w.r.t SPECIFIC vantage points and then at query time we exploit triangular inequality properties to prune the search space according to the distance between the user query point and the search radius (for range queries)

Consider a range query $\{p: D(p,q) \leq r\}$.
If $D(q,v) > r_v + r$ we can conclude that
no point in $P1$ belongs to the result

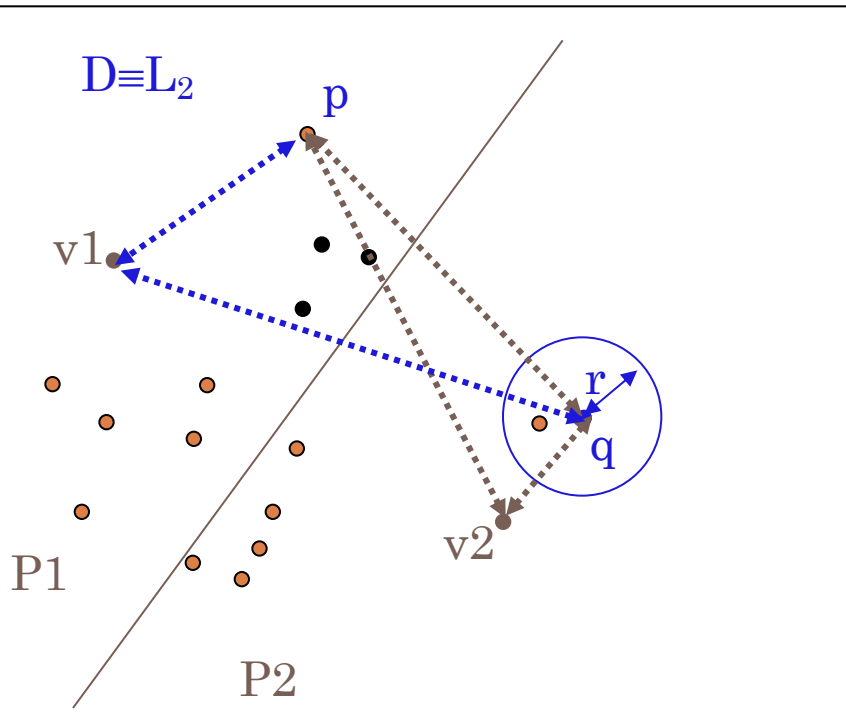
Similar arguments apply to $P2$

Principles of metric indexing (2)

Generalized Hyperplane: take two points $v1$ and $v2$, compute the distances of all other points p w.r.t. $v1$ and $v2$, and define

$$P1 = \{p : D(p,v1) \leq D(p,v2)\}$$

$$P2 = \{p : D(p,v2) < D(p,v1)\}$$



Consider a range query $\{p: D(p,q) \leq r\}$.
If $D(q,v1) - D(q,v2) > 2*r$ we can
conclude that no point in P1 belongs
to the result

The M-tree



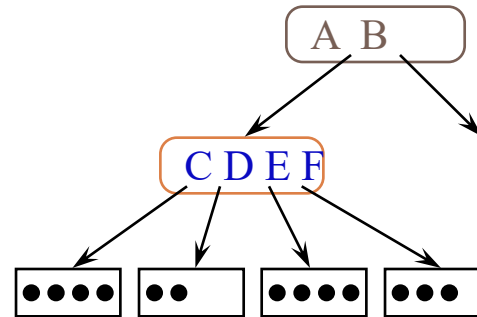
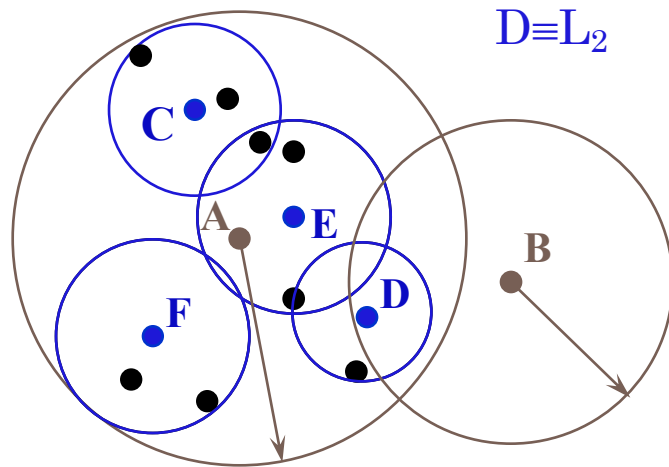
- The M-tree has been the first dynamic, paged, and balanced metric index
- It generalizes R-tree principles to arbitrary metric spaces
- Since its introduction, the M-tree has been used by several research groups for image retrieval, text indexing, shape matching, clustering algorithms, fingerprint matching, DNA DB's, etc.
 - More than 2,600 citations on Google Scholar
- C++ source code freely available at <http://www-db.disi.unibo.it/Mtree/>
- At a first sight the M-tree “looks like” an R-tree.
- However, the M-tree only “knows” about distance values, thus it ignores coordinate values and does not rely on any “geometric” (coordinate-based) reasoning

the Marco-tree LOL

Paolo Ciaccia, Marco Patella, Pavel Zezula.

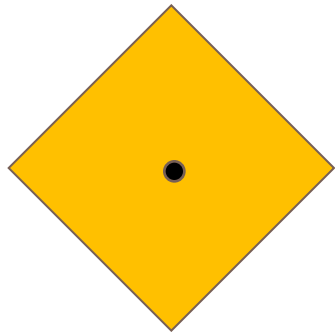
M-tree: An Efficient Access Method for Similarity Search in Metric Spaces. VLDB 1997: 426-435

M-tree: how it looks like

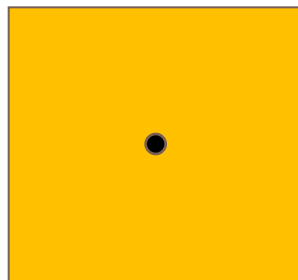


- Recursive bottom-up aggregation of objects based on regions
- Regions can overlap
- Each node can contain up to C_{max} entries, but not less than $C_{min} \leq 0.5 * C_{max}$
 - The root makes an exception

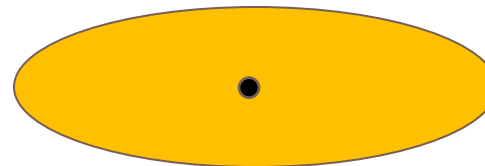
Depending on the metric, the “shape” of index regions changes



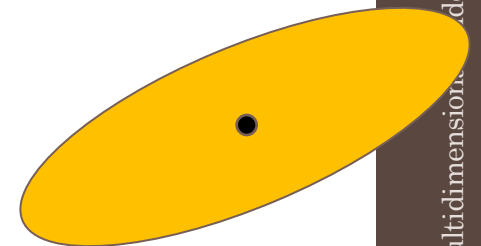
L_1



L_∞



Weighted Euclidean



quadratic distance

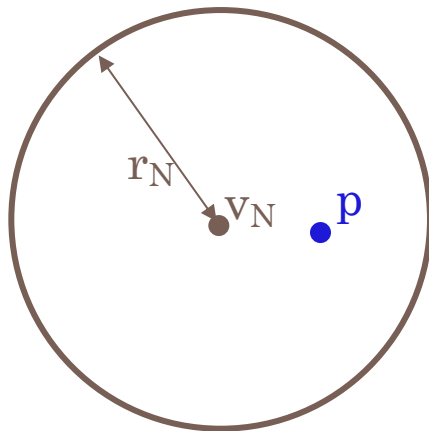
The M-tree regions

- Each node N of the tree has an associated region, $\text{Reg}(N)$, defined as

$$\text{Reg}(N) = \{p: p \in U, D(p, v_N) \leq r_N\}$$

where:

- v_N (the “center”) is also called a **routing object**, and
- r_N is called the **(covering) radius** of the region
- The set of indexed points p that are reachable from node N are guaranteed to have $D(p, v_N) \leq r_N$



This immediately makes it possible to apply the pruning principle:

If $D(q, v_N) > r_N + r$ then prune node N

Entries of leaf and internal nodes

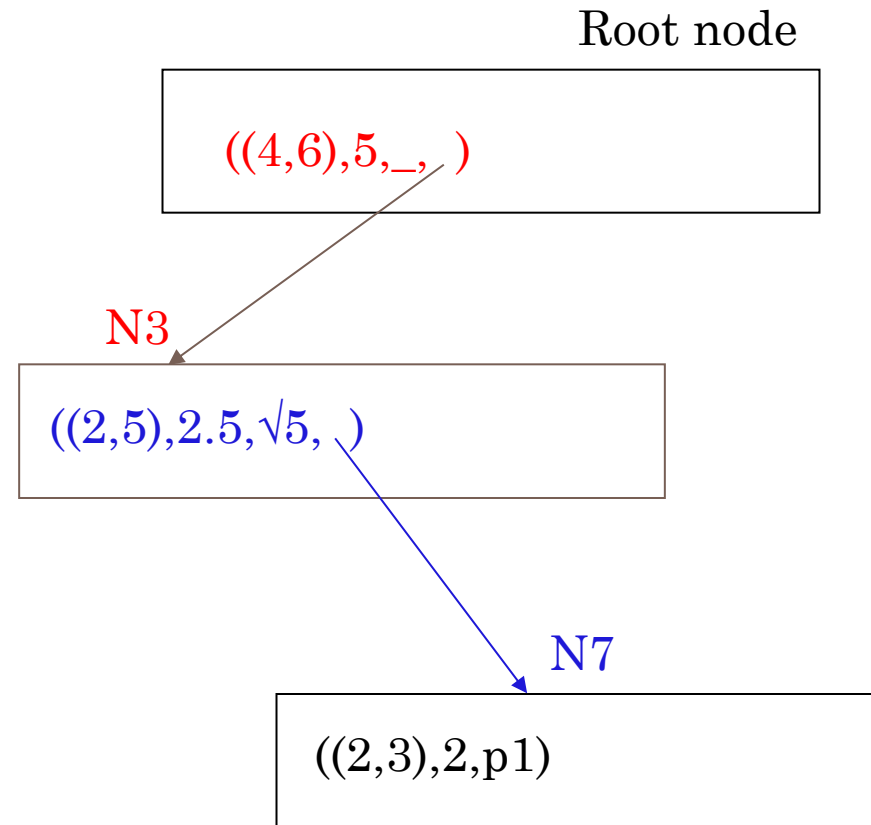
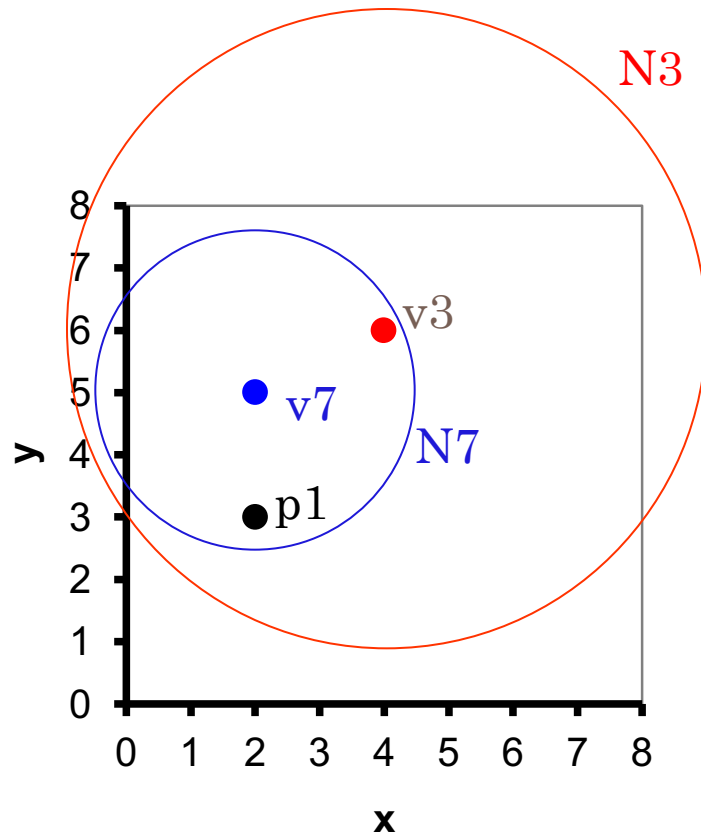
Leaf node:

- An entry E in node N has the form $E = (\text{ObjFeatures}, \text{distP}, \text{RID})$, where:
 - ObjFeatures are the feature values of the indexed object
 - distP is the distance between the object and its parent routing object (i.e., the routing object of node N)

Internal node:

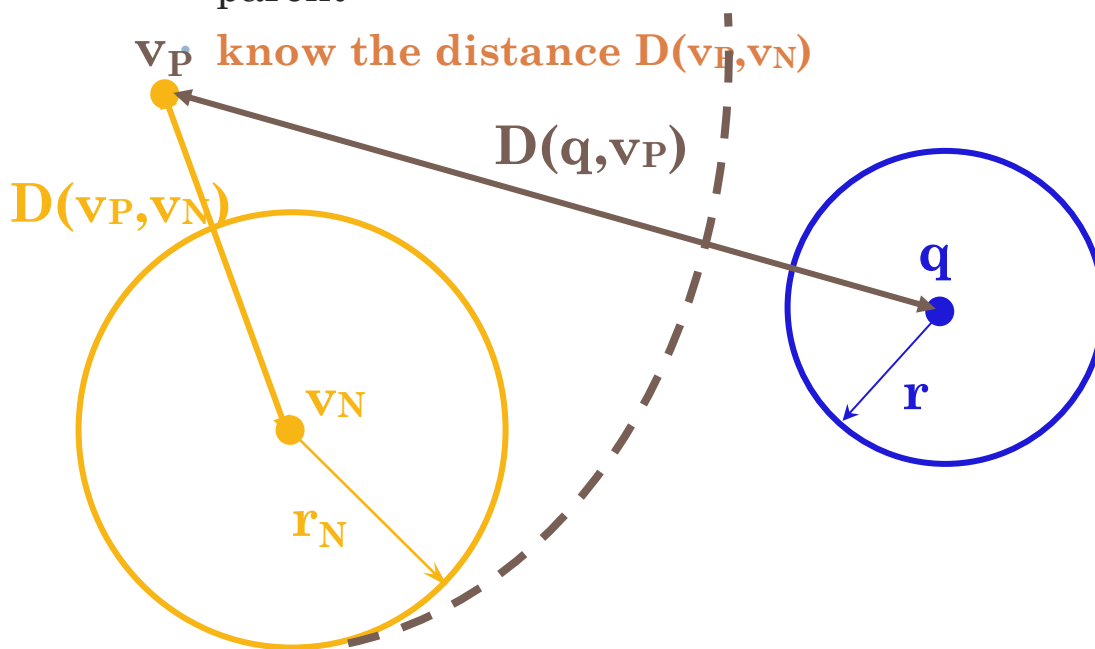
- $E = (\text{RoutingObjFeatures}, \text{CoveringRadius}, \text{distP}, \text{PID})$, where
 - RoutingObjFeatures are the feature values of the routing object
 - CoveringRadius is the radius of the region
 - distP is the distance between the routing object and its parent routing object (undefined for entries in the root node)

Entries: an example



Fast pruning based on distP

- **Pre-computed distances distP** are exploited during query execution to save distance computations
 - A major goal for metric indices, since distance computation can be costly!
- Let v_P be the parent (routing) object of v_N
- When we consider the entry of v_N , we
 - have already computed the distance $D(q, v_P)$ between the query and its parent



From the triangle inequality:

$$D(q, v_N) \geq |D(q, v_P) - D(v_P, v_N)|$$

Thus we can prune node N *without computing $D(q, v_N)$* if

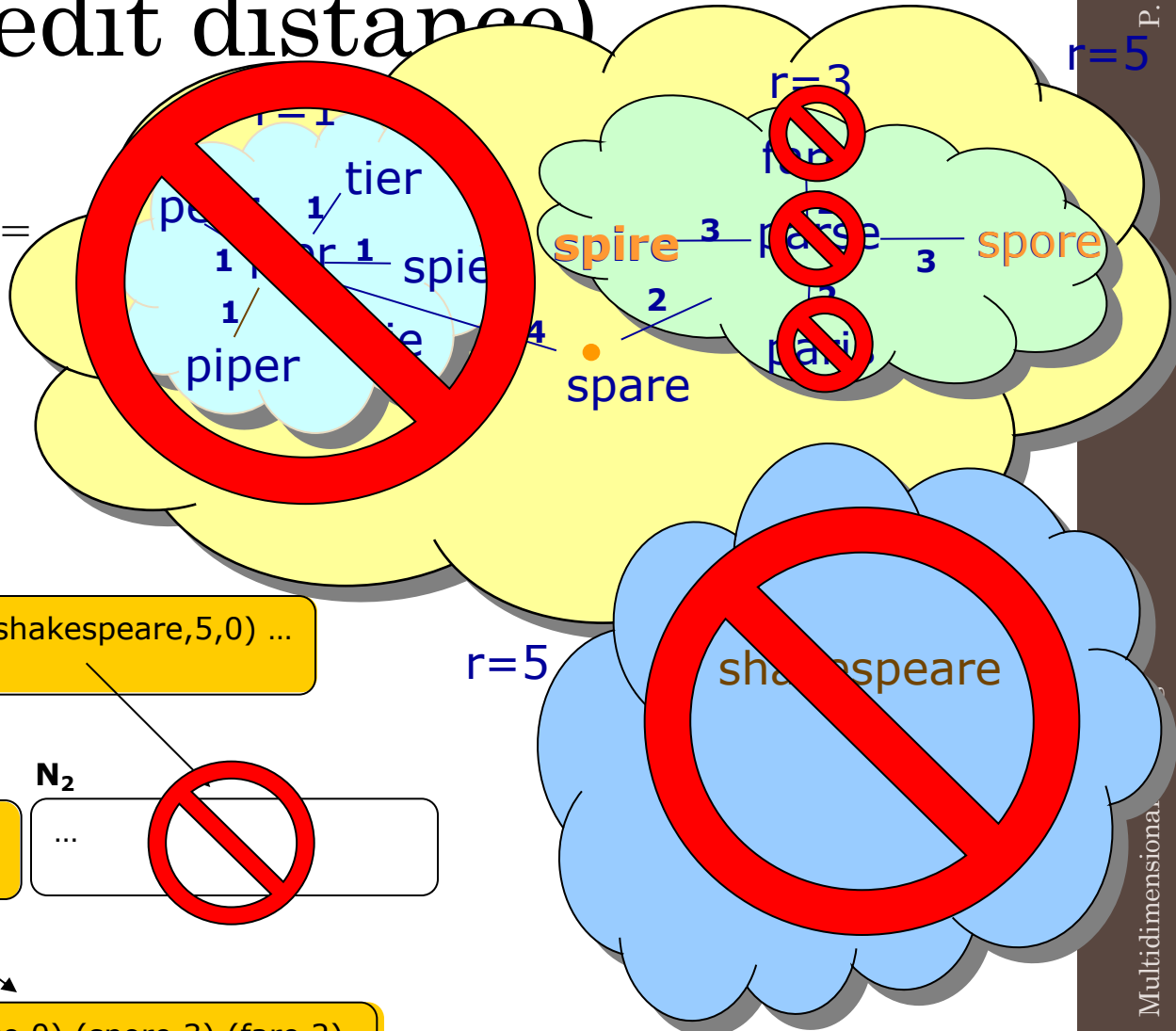
$$|D(q, v_P) - D(v_P, v_N)| > r_N + r$$

Example (edit distance)

query = "spire", $r = 1$

$d(\text{"spire"}, \text{"shakespeare"}) = 3 \leq 3 + 1$

$|d(\text{"spire"}, \text{"spare"}) - d(\text{"spare"}, \text{"shakespeare"})| = |3 - 0| = 3 \leq 3 + 1$



N_0

(spare,5,0), (shakespeare,5,0) ...

N_1

(pier,1,4) (parse,3,2) ...

N_2

...

N_3

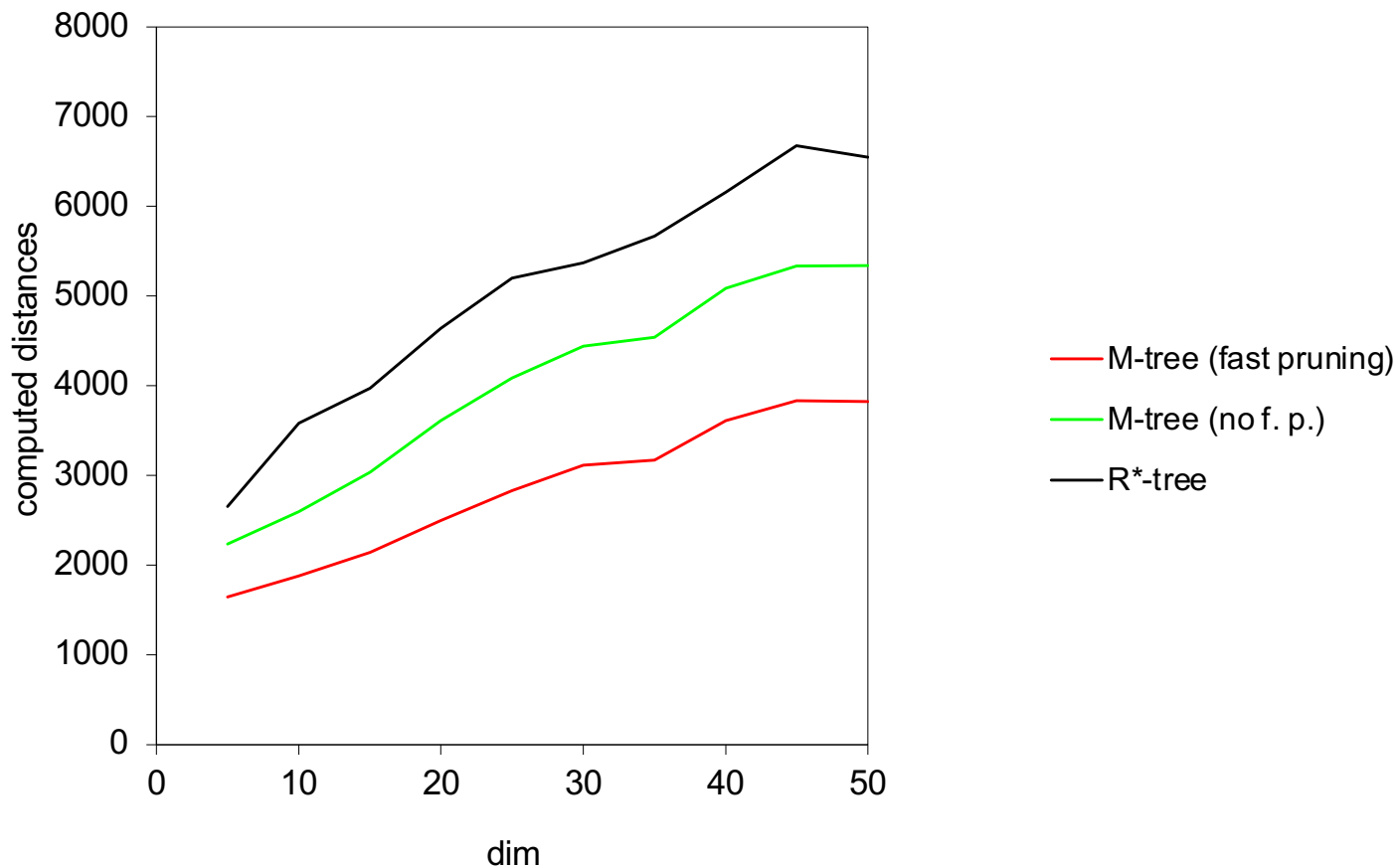
(pier,0) (pie,1) (piper,1) (piper,1)

N_4

(parse,0) (spore,3) (fare,2) (spire,3) (paris,2)

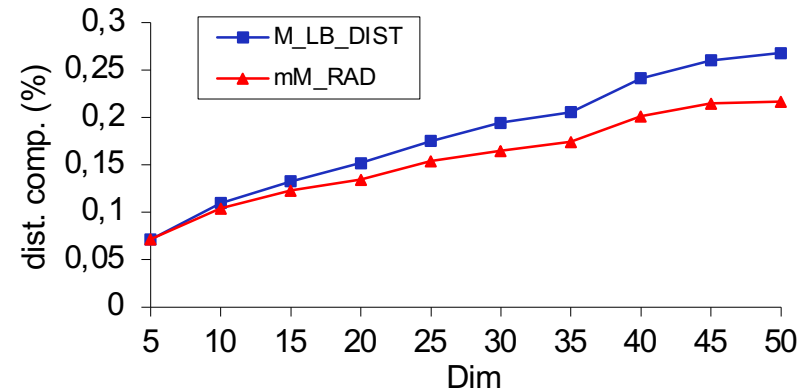
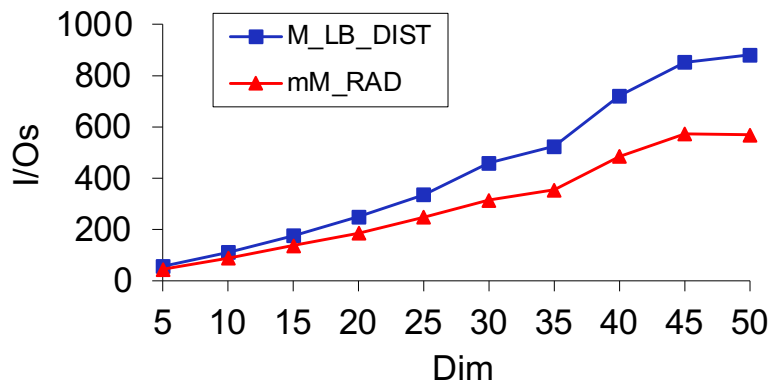
Experimental results

- Synthetic datasets (10 Gaussian clusters)
- Up to 40% cost reduction with fast pruning



Insertion and split (sketch)

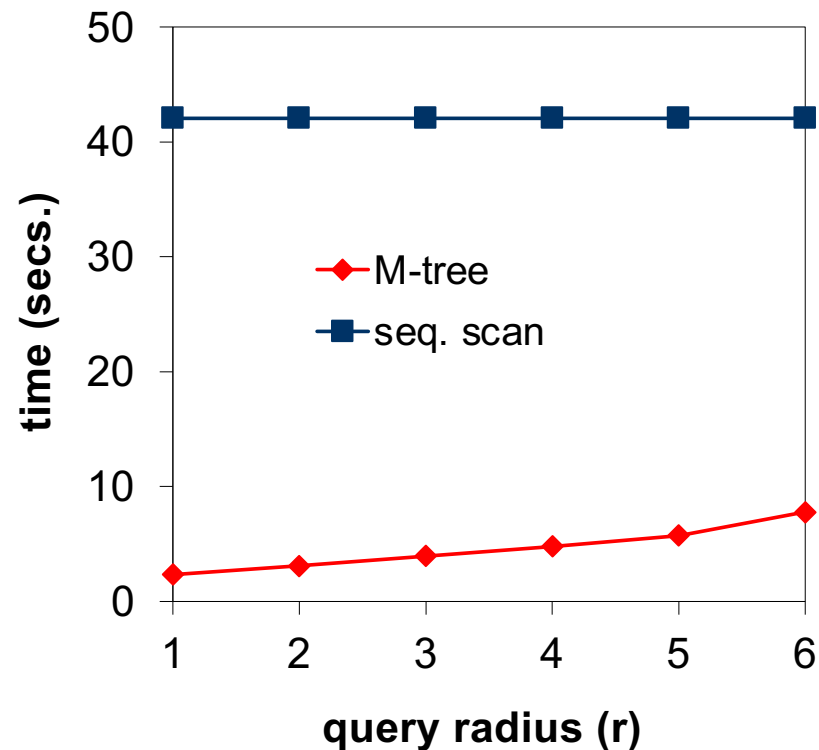
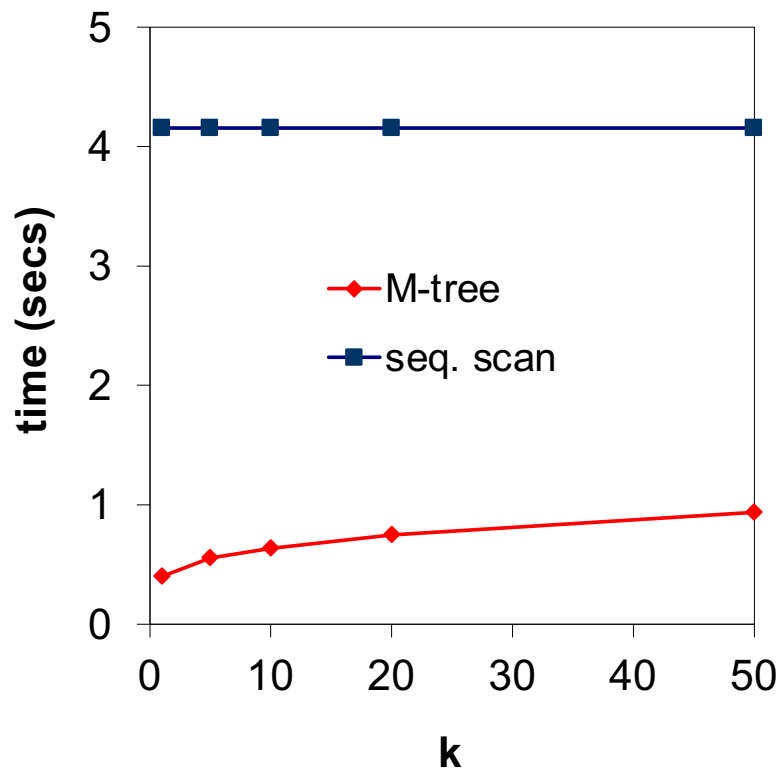
- The procedure to insert a new object is based on a Penalty method
- The Penalty method considers the increase of the covering radius needed to accommodate the new object
 - Remind: no “volume” can be computed!
- For managing a split, there are several alternatives, among which:
 - mM_RAD minimize the maximum of the two resulting radii
 - M_LB_DIST choose the closest and the farthest object from v_N
- Experiments demonstrate that mM_RAD is the best



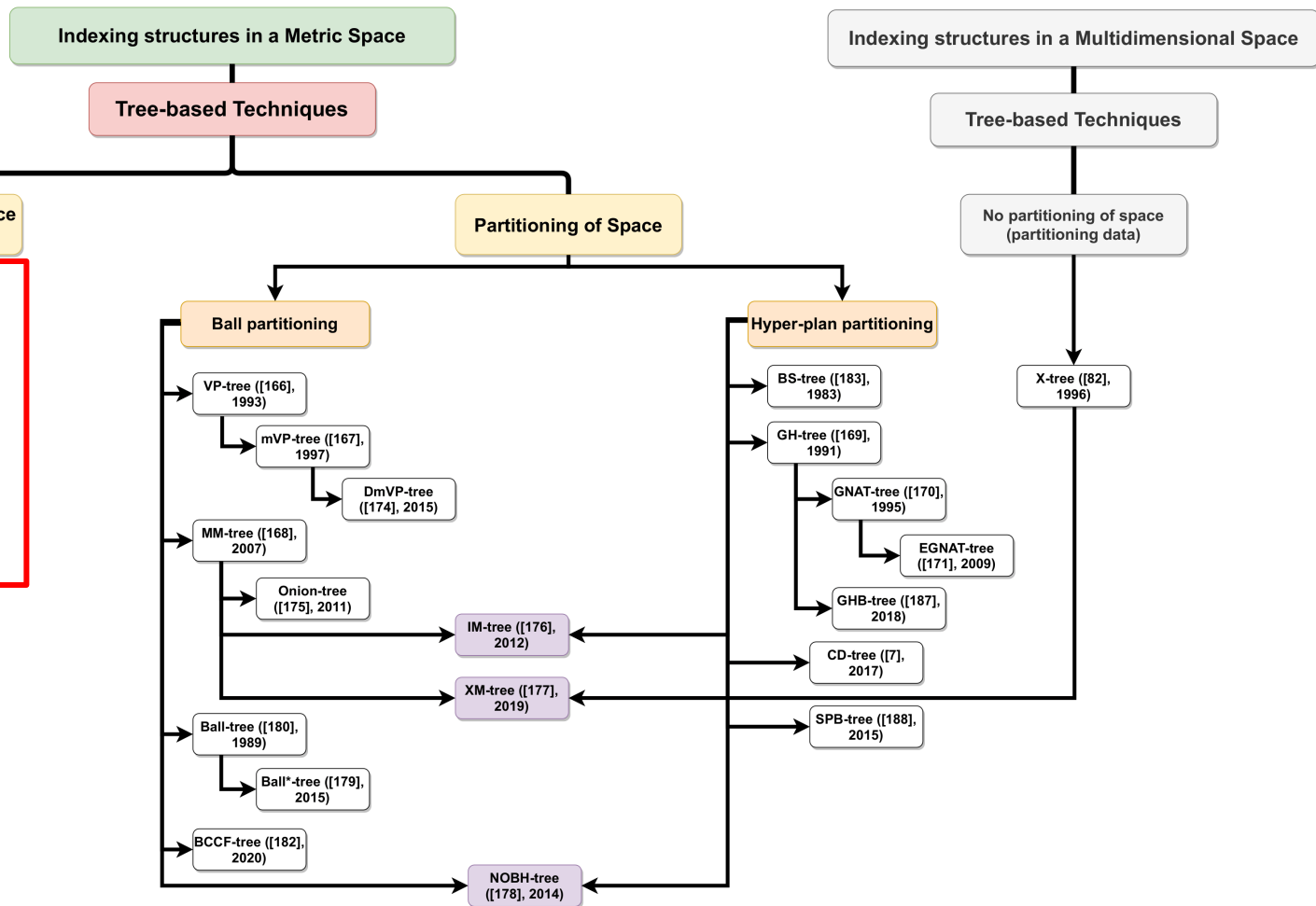
Experiments (k-NN and range q.)

- 68,000 color images
- 32-dim (color histograms), L_2
- 161,212 text rows
- Edit distance

The logic of search algorithms is the one already seen for range and k-NN queries with the R-tree



Other metric indices



What's next?

- Spatio-temporal data
 - Moving objects
 - Time series
- (Very) high-dimensional data
 - Approximate k-NN queries
 - Dimensionality reduction
- Other query types
 - Skyline & F-skyline queries
 - Regret queries
- Distributed scenarios
- ... and many other issues